

Ninth Gen to Second Gen: Adapting a Modern Game to 8-bit

Matthew Prickett

2212805

Project Dissertation



Swansea University
Prifysgol Abertawe

Department of Computer Science

Adran Gyfrifiadureg

26th July 2025

Declaration

Statement 1

This work has not been previously accepted in substance for any degree and is not being concurrently submitted in candidature for any degree.

Signed Matthew Prickett (2212805)

Date

Statement 2

This thesis is the result of my own investigations, except where otherwise stated. Other sources are acknowledged by citations giving explicit references. A bibliography is appended.

Signed Matthew Prickett (2212805)

Date

Statement 3

The University's ethical procedures have been followed and, where appropriate, ethical approval has been granted.

Signed Matthew Prickett (2212805)

Date

Abstract

Consoles are becoming ever more powerful and games ever larger and ambitious, but the humble beginnings of console gaming were far more limiting. In 1977, Atari released the Atari 2600 with only 128 bytes of RAM and 4K ROM. Its fundamental hardware design was specifically meant to interface with CRT-TVs using its TIA chip to recreate hit arcade games at the time, like Pong and Tank. Despite its limitations though, talented developers have created some of the most influential games of all time on the system, with more still making new games for the console to this day. Being an exercise in low level programming, utilising 6502 assembly, this project creates a piece of software that interacts directly with the hardware itself with minimal abstraction, building upon existing work to create a new game for the console. Inspired by ‘Halo 2600’ by Ed Fries, ‘Power Wash Simulator 2600’ is an 8-bit recreation of the popular modern video game, ‘Power Wash Simulator’ by FuturLab and Square Enix for the Atari 2600. Players are able to blast away dirt, play with their friends and express their creativity all in the space of a 2KB ROM, whilst maintaining that sense of fun. Development for such a limited system highlights the interaction between software and hardware that is abstracted in higher level languages, inspiring creative solutions to navigate challenges that exist only at this level. Reaching wider, an understanding of this interaction could help improve the efficiency of modern software applications, especially games, to make better use of the hardware capabilities, or in systems where memory and other resources are far more expensive, possibly in handheld systems.

Acknowledgements

First and foremost, this project would like to thank Dr Benjamin Mora for his supervision and providing invaluable support and feedback throughout development. Thank you so much for suggesting the project too, this project, despite its challenges, has been a joy to work on.

Thanks also goes to Professor John Tucker for taking a keen interest in the project and offering additional valuable advice, appreciating the history of the console and its impact on society.

Special thanks goes to Megan Venn-wyherley for organising the 'CoFo Goes Gaming event' and allowing others to view the game in progress and give feedback.

A big thanks goes to Dr Sean Walton for organising the Video Games showcase event, allowing even more people to view the game at the end of its development, providing great insights. Thank you to everybody who came along and gave the game a go!

Finally, a huge thank you to all the friends and family who supported this project in so many ways from even before it began, offering encouragement even when things were tough.

The game would not be what it is without all of you! Thank you!

Contents

1	Introduction	1
1.1	Aims and Objectives	1
2	Background	3
2.1	Atari and the VCS	3
2.2	Console Design and Specification	4
2.3	Limitations of the Atari 2600	5
2.3.1	Sprites	5
2.3.2	'Pixels' and Resolutions	6
2.3.3	Racing the Beam	6
2.3.4	Scanlines and Machine Cycles	7
2.3.5	ROM Space	8
2.4	Power Wash Simulator	8
3	Learning	9
3.1	Reading Books and Setting Goals	9
3.2	A Brief Introduction to 6502 Assembly	10
3.3	Daily Logging	11
4	Development	12
4.1	Tools	12
4.2	Prototyping	12
4.2.1	Cleaning	12
4.2.2	Two Player Movement	14
4.2.3	Scoring	15
4.2.4	Shooting	16
4.3	Becoming Lost in Code and Doubts	17
4.4	Playing with Playfield Graphics	18
4.4.1	Getting Dirty	18
4.4.2	Drawing Dirt	19
4.4.3	Cleaning Dirt	20
4.5	Aiming	21
4.5.1	Intuitive Controls	21
4.5.2	Reticles and Bugs	22
4.5.3	Going Beyond the Beam	24
4.6	Making a Game	24
4.6.1	Winning	24
4.6.2	Calculating Percentages	25
4.6.3	Random Numbers and Procedural Generation	27
4.6.4	Animations, Sprites and Audio	28
4.7	Publishing a Game onto a Physical Medium	29
5	Testing	29
5.1	Functional Requirements	29

5.2	Non-functional Requirements	30
6	Reflection	30
6.1	The Process and Improvements	30
6.2	The Product	31
7	Conclusion	32

List of Figures

1	A picture of an original Atari VCS system [13]	3
2	A high level diagram of the Atari 2600 hardware [23]	4
3	A diagram showcasing the vertical blank, horizontal blank and overscan periods, measured in scanlines in machine cycles [23]	7
4	Box art from 'Power Wash Simulator' [30]	8
5	A screen capture of the prototype cleaning function built upon the original code by Steven Hugg [18]	14
6	A prototype version of the players being able to move and shoot with visual glitches.	16
7	The prototype build presented at the Student Conference, showing player movement, scoreboard and firing	17
8	A screenshot of the asymmetrical playfield graphics used for the dirt. Missiles are absent.	19
9	A screenshot showing the early dirt drawing and washing	20
10	A screen capture of the control scheme for a gamepad controller in Power Wash Simulator[38].	21
11	A screenshot of the completed drawing, with reticles and 72 dirt segments.	25
12	A screenshot of the final completed game in play	28
13	A screenshot showing a display issue after changing the kernel to a two-line kernel.	35
14	An error message from the IDE stating that the jump distance has exceeded 128 bytes	35
15	A screenshot of an early version of pseudo-random generation using the game code, eventually leading to procedural generation	36
16	A screenshot showing the issue where the last three bits on the left side were repeating the right side	36
17	A screenshot showing P0 going no further than the left edge.	62
18	A screenshot showing P0 going left.	62
19	A screenshot showing no input for either player.	62
20	A screenshot showing P0 going right.	63
21	A screenshot showing P0 going no further than the right edge.	63
22	A screenshot showing P1 going no further than the left edge.	63
23	A screenshot showing P1 going left.	64
24	A screenshot showing P1 going right.	64
25	A screenshot showing P1 going no further than the right edge.	64
26	A screenshot showing P0 reticle going no further top of play area.	65

27	A screenshot showing P0 reticle going up.	65
28	A screenshot showing P0 reticles not moving with no input.	65
29	A screenshot showing P0 reticle going down.	66
30	A screenshot showing P0 reticle going no further then the bottom of the play area.	66
31	A screenshot showing P1 reticle going no further top of play area.	66
32	A screenshot showing P1 reticle going up.	67
33	A screenshot showing P1 reticle going down.	67
34	A screenshot showing P1 reticle going no further then the bottom of the play area.	67
35	A screenshot showing neither player firing their washer.	68
36	A screenshot showing Player 0 firing their washer.	68
37	A screenshot showing Player 1 firing their washer.	68
38	A screenshot showing neither players in firing mode and causing a collision.	69
39	A screenshot showing Player 0 firing and causing a dirt piece to be washed.	69
40	A screenshot showing Player 0 firing and not causing a dirt piece to be washed.	69
41	A screenshot showing Player 0 not firing and not causing a collision.	70
42	A screenshot showing Player 1 firing and causing a dirt piece to be washed.	70
43	A screenshot showing Player 1 firing and not causing a dirt piece to be washed.	70
44	A screenshot showing Player 1 not firing and not causing a collision.	71
45	A screenshot showing every row and column can be washed.	71
46	A screenshot of the complete test showing score is only added when a whole playfield segment is cleaned.	71
47	A screenshot showing one dirt piece left and the game continuing	72
48	A screenshot showing 40 dirt pieces left and the game continuing	72
49	A screenshot showing the P0 win animation	72
50	A screenshot showing the P1 win animation	73
51	A screenshot showing random loading when in BW mode and P1 expert mode	73
52	A screenshot showing the colour screen for area two	73
53	A screenshot showing the colour screen for area three	74
54	A screenshot showing the colour screen for area four	74
55	A screenshot showing procedural generation test case 1	74
56	A screenshot showing procedural generation test case 2	75
57	A screenshot showing procedural generation test case 3	75
58	A screenshot showing procedural generation test case 4	75
59	A diagram of the Playfield Graphics, showing that only 20 bits store the data. These can be repeated or mirrored [23]	77

List of Tables

1	The number of scanlines and machine cycles non-display periods last.	7
2	Example showing relevant addressing modes of the instruction LDA and the computational cost associated with it.	11
3	Example of using different symbols to indicate different bases.	11
4	Comparison of code solutions to save machine cycles	23
5	Machine cycle cost of each part of algorithm to calculate and display percentage.	26
6	An example of how data may be split over three registers for two images.	27

7	Test Suite for Player 0 movement	54
8	Test Suite for Player 1 movement	54
9	Test Suite for Player 0 aiming	55
10	Test Suite for Player 1 aiming	55
11	Test Suite for Player 0 Shooting	56
12	Test Suite for Player 1 Shooting	56
13	Test Suite for Player 0 Cleaning	56
14	Test Suite for Player 1 Cleaning	57
15	Test Suite for cleaning whole segments of rows and columns (see Figure 45 for complete test).	57
16	Test Suite for adding score to the player who cleaned the last bit in a segment (see Figure 46)	58
17	Test Suite for checking that all segments are cleaned (TestAllClean subroutine)	58
18	Test Suite for winning	58
19	Test Suite for testing that the ding sound audio plays when only when either player clears a segment.	58
20	Test Suite for switching between different load modes	59
21	Test Suite for changing colours every new round	59
22	Test Suite for looking up 8-bit segment procedural loading	59
23	Test Suite for looking up 4-bit segment procedural loading	59
24	Test Suite for procedural generation loading	60
25	Test Suite for Player 1 movement animation	60
26	Test Suite for Player 1 movement animation	60
27	Test Suite for shooting audio	61
28	Test Suite for game win music jingle	61

1 Introduction

1.1 Motivation

Releasing in 2020, the Xbox Series X and PlayStation 5 are part of the ninth generation of consoles, alongside the new handheld PCs, such as the Steam Deck, and the Nintendo Switch [28]. A comparison of the Xbox Series X and PlayStation 5 reveals just how powerful these systems are, each having 16GB of RAM, eight core CPUs at 3.5GHz or more clock speeds, and impressive graphical capabilities, outputting a resolution of 4K or even 8K [25].

Utilising game engines, such as Unity or Unreal Engine, game development has plenty of powerful tools to allow developers to create whatever they can imagine across a variety of domains, including animation, physics, audio and more [40]. With so much power and freedom these consoles and tools allow developers, it is important to appreciate where these originated from and the constraints developers for much older systems had to go against to produce many of the incredible games that inspired those we see today.

Video games have their origins in the 1940s with machines that could play games like Nim [36]. Moving into the 1950s and 1960s, work on similar machines continued, including the creation of a tennis-like game in 1958 by Willy Higinbotham, often argued to be the first ever video game [37]. His work would go onto inspire the games on a prototype by Ralph Baer that could plug into a TV at home, known as the 'Brown Box' in 1967, becoming the Magnavox Odyssey in 1972, starting the first generation of home consoles [36]. Later, in 1975, Atari released a console called 'Home Pong', able to play a version of their popular arcade machine 'Pong'. Building on from its success, Atari released their second-generation console in 1977, the Atari Video Computer System (VCS), that would later become known as the Atari 2600 [10].

Preserving the console long past its initial life, new games are even being developed and published by the homebrew community on sites such as AtariAge [12]. Building upon the work by developers old and new, and as a token of appreciation to the system that had such an impact on modern gaming, this work goes through the journey of creating an adaptation of 'Power Wash Simulator' [30], a ninth-generation game, for the second-generation console, detailing its development, testing and ability to be played on actual hardware. While such a task was not easy, this project hoped to reveal how capable old hardware can still be and the lessons that can still be learned from these systems, critiquing modern video game creation and the freedoms allowed to it. Additionally, it hoped to apply research suggesting constraint may encourage creativity [31] in the context of video game creation, as developing a game for the system presents many limitations and challenges, many of which are described in this work. Necessitating that unique and creative solutions be found, it was also an opportunity for the developer to explore working in a low-level environment with an unfamiliar programming language, hopefully gaining a better appreciation for the underlying systems and hardware modern languages often abstract, gaining new skills that could be taken to new contexts, such as machine learning [22].

1.2 Aims and Objectives

In order to produce a good quality game for an 8-bit console, five aims were established. Objectives provide more specific insight into how these aims were to be achieved.

1. To target a piece of hardware that has limited resources, including memory and computational power.
 - (a) Learn suitable low-level languages that can be used to create software for the given hardware system.
 - (b) Program a piece of software using low-level languages to effectively manage memory and other low-level systems, such as input/output.
2. To adapt modern gameplay mechanics and controls to an 8-bit console:
 - (a) Identify core gameplay features of the original features of the original game and incorporate them into the 8-bit version, keeping them recognisable.
 - (b) Translate modern control schemes to retro controllers such that they are intuitive to pick up and play.
 - (c) Explore the functionality of the console hardware and draw inspiration from related work to provide additional gameplay features that may not have existed in the original game.
 - (d) Create an engaging gameplay loop such that players desire to play beyond one play session.
3. To reasonably maintain modern graphical style and design in an 8-bit environment:
 - (a) Identify recognisable graphical elements, objects, art and themes (such as colour palettes and aesthetic design) belonging to the original game.
 - (b) Explore techniques for navigating the graphical limitations of the hardware.
 - (c) Implement graphics using techniques that best suit the hardware to create a visually interesting game that can be identified as deriving from the original game.
4. To employ modern software and game development practices to produce an 8-bit game:
 - (a) Research and make use of modern software and tools designed for the system, such as debuggers, to aid the development practice.
 - (b) Employ the use of modern software development practices, such as scrum, to deliver and produce software deliverables in a timely fashion.
 - (c) Follow the development lifecycle of a modern game.
5. To publish the game to a physical medium:
 - (a) Explore options to place the game on a physical medium that the original hardware system can read from.
 - (b) To have a playable ROM that can be inserted into and ran on the original hardware.

2 Background

Before delving into development of the game, it is important that the hardware and context of the Atari 2600 is explained as many of the challenges presented are unique to the hardware, deriving from its design, development and historical context.

2.1 Atari and the VCS



Figure 1: A picture of an original Atari VCS system [13]

Atari was founded by Nolan Bushnell and Ted Dabney in June 1972. Soon after, the arcade machine ‘Pong’, developed by their employee, Alan Alcorn, was released [10]. Atari would continue to develop their arcade machine line-up and explore taking their games to the home console market with their Home Pong system in 1975 [36]. Rather than designing a new system for each game, their second-generation machine, the Atari Video Computer System (VCS), was conceived and designed with two goals in mind: It should be able to imitate already existing and successful games, such as Pong; and it should have some flexibility for new games to be programmed for the system [23]. In 1977, the Atari VCS, later known as the Atari 2600 was released [10].

Despite releasing after the Fairchild Channel F, the Atari 2600 dominated the market by 1982 [23] and had around 30 million units sold throughout its lifetime [29], while other consoles in the generation, including the Magnavox Odyssey 2 (1978), Intellivision (1980), ColecoVision (1982), and the Atari 5200 (1982), were unable to match the Atari 2600 success despite being technically superior [13]. In general, second-generation consoles could be characterised to have the following properties: better resolutions, three- or eight-bit colours, up to three channels for audio, multiple playfields spanning multiple screens and, most notably, a microprocessor, allowing the consoles to accept cartridges (or ROMs) containing the game code [13]. The generation saw rapid development, with greater RAM and ROM capacity, processors (the IntelliVision had a comparatively more powerful 16-bit CPU), and graphical capabilities (the ColecoVision had near arcade quality graphics) [36]. The Atari 2600 was eventually dethroned after the video game crash in 1983 [23] by the Nintendo Entertainment System (NES) which was released in the same year, beginning the third generation of consoles [36].

2.2 Console Design and Specification

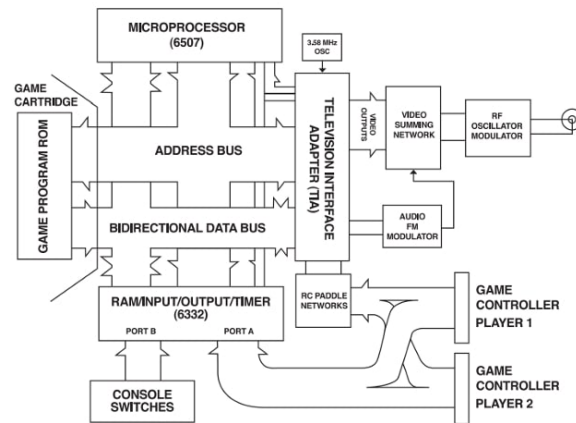


Figure 2: A high level diagram of the Atari 2600 hardware [23]

Popular computers at the time, such as the Apple II and the Commodore 64, and even the NES, were powered by the attractive inexpensive-but-powerful eight-bit MOS Technology 6502 processor [32]. While the 6502 was low-cost, it was still too expensive for the Atari 2600 to use, so an even lower cost variant of the processor, the 6507, was used instead [16].

At a clock speed of 1.19 MHz, the 6507 has 28 pins, much fewer than the 6502 [16]. Cutting the number of address pins from 16 to 13, this limits the addressable memory space from 64K to 8K, and out of that, 4K is reserved for addressing system I/O and RAM, while the remaining is used to access the ROM (i.e. the game code on the cartridge). The remaining pins are used for data (eight pins), power, resetting, the clock and reading/writing signals [1].

With a look at a comprehensive specification sheet for the system [6], the 6507 was not the only chip on the system. Accompanying it was the MOS Technology 6532 chip, or also known as the RAM/Input/Output/Timer (RIOT), or Peripheral Interface Adapter (PIA) in the Stella's Programming Guide [39]. Being appropriately named, the RIOT chip held the only random-access memory (RAM) on the whole system; a mere 128 bytes. For comparison, that is roughly 125 million times less space compared to modern consoles.

Additionally, the RIOT chip handled input from the controllers, most notably the joystick [23]. Input via the various switches on the console itself, such as the left and right difficulty selectors (P0 and P1 expert mode switches), colour or black and white switch (BW switch), and reset button provide other ways for the player to interact with the platform [23]. Other responsibilities of the chip were to synchronise program execution using its timer and handle output [39].

Being custom built for the system, the Television Interface Adapter (TIA) is the third chip on the device, responsible for outputting the graphics to the TV, specifically a Cathode Ray Tube (CRT) TV, common in 1970s and 80s [23]. The TIA made the Atari 2600 unlike its competition and even modern consoles, which use more standard frame buffers to display graphic information [23]. Hardware features of the TIA made some aspects of game development trivial, having hardware features that enable a developer to easily flip sprites or detect collisions [39]. Despite

this, among other limitations such as the RAM size, the TIA is notorious for presenting many of the challenges that come with developing for the system [23].

2.3 Limitations of the Atari 2600

An in-depth study of the system, by Ian Bogost and Nick Montfort in their book called ‘Racing the Beam’ [23], provides a great insight into the development of the Atari 2600 and the many limitations of it, while also exploring the creative solutions Atari and third-party developers, such as Activision, came up to navigate them. The Stella’s Programming Guide by Steve Wright [39] also details the various hardware features and functionalities of the system the developer can use. Highlighted in this section are the most prominent challenges that are often encountered during the development process, including for this project.

2.3.1 Sprites

In comparison to later systems like the NES, which could hold multiple sprites and tiles [36], the Atari 2600 can hold a mere five [23]. The Atari 2600 launched with a select few games, but its development was largely influenced by two of Atari’s most popular arcade machines: ‘Pong’ and ‘Tank’ [23]. Indeed, the fundamental design of the TIA’s features are specifically for these games. For the adaptation of ‘Tank’, known on the Atari 2600 as ‘Combat’, the TIA was designed to store data for two player sprites and two missile sprites [23]. In the game, the player sprites are the tanks used for either player (or perhaps Biplanes or Jets in the other ‘games’ included on the cartridge), and the missiles are shot out from the parent tank (missile and player sprites are inherently linked [39]). According to ‘Racing the Beam’ [23], ‘Combat’ is very much a showcase of what is possible on the system, but only having two player sprites is very limiting to adapt a modern game, or even for adapting an arcade game from the 80s. Fortunately, hardware features alleviate some of the constraint of having only two player sprites, as the player sprites can be repeated.

In Combat’s biplanes or jets games, the player sprites are repeated to show three copies of the plane appearing to fly in unison [23]. Another example can be seen in ‘Space Invaders’, an adaptation of the arcade machine of the same name, where the approaching lines of aliens are really one sprite repeated across the screen [23]. A limitation of this is that it is the same sprite, whereas having unique sprites would be useful. Luckily, an unintended quirk of the hardware allows for just this.

While not used in ‘Combat’, other games use this feature alongside the register to horizontally move a sprite, HMOVE. By ‘strobing’ this register (meaning simply writing to the register; the data is ignored), and updating the sprite data will create the illusion of more unique sprites being displayed horizontally on the screen, despite all technically being the same sprite [39]. This is used to great effect in games like ‘Air-Sea-Battle’, where multiple horizontally aligned and unique ships are displayed [23]. Unfortunately, the same technique cannot be used vertically. Consequently, games like ‘Pac-Man’ suffer from four flickering ghosts, which move both horizontally and vertically, becoming quite an eye-sore. This effect was somewhat mitigated on a CRT TV [23], whereas using more modern screens makes the effect more evident.

To accommodate ‘Pong’, an additional sprite called the ‘Ball’ was added to the design of the

TIA [39]. While it may be hoped that the two missile and ball sprites could be used for more complex graphics, this is not the case: they are only one bit wide. In comparison, player sprites are eight bits. On the other hand, they can be used creatively for other purposes than what they originally were used for. In Adventure, for example, the ball is used for the player, reserving the player sprites for items, and the missiles are used for thin walls [23]. As a result, the items are detailed, but the player is just a dot on screen. A more creative use is seen in ‘Pitfall!’ where the ball is used to create a rope that swings.

2.3.2 ‘Pixels’ and Resolutions

Touched upon earlier, player sprites are eight bits wide. However, that is not to say that they are only eight pixels wide. A better term is a ‘clock cycle’, referring to the clock inside the TIA, and appears more rectangular in shape, and is only one scanline high [23]. More generally, it may be useful to think of this as the resolution of the sprites, as the player sprites can be scaled to some extent, either double, quadruple or eight times the size, using the NUSIZx registers [39]. Similarly, the ball and missile sprites can be scaled too. Player sprites are the highest resolution object available. Missiles and the ball are less useful but can be used for some detail as discussed in some earlier examples. Exhausting those, playfield graphics are used instead, especially for larger objects.

Playfield graphics are commonly used for walls, floors, or less detailed background objects because the TIA can easily test for collisions between playfield graphics and sprites [23]. On the Atari 2600, the maximum resolution is 160x192, that is 160 clock cycles horizontally and 192 scanlines vertically [39]. A single playfield graphic is four clock cycles wide, meaning there can only be 40 playfield graphics across the screen. However, there are only 20 bits to hold playfield graphics data stored across three eight-bit registers (known as PF0, PF1 and PF2), enough for only half the length of one scanline (see Figure 59) [39]. To solve this issue, the TIA provides the functionality to mirror or repeat this data to the other half of the screen [39]. This allows for horizontally symmetrical arenas to be created for example, although it is possible for asymmetrical data to be displayed too through careful timing.

2.3.3 Racing the Beam

As mentioned previously, the TIA is designed to interface with CRT TVs. Inside the TV, the electron gun directs a beam of electron across the screen from left to right as we see it, displaying the graphics for one scanline. Once it has reached the edge of the screen, the electron gun will then return to the left side to then display the next scanline, known as the horizontal blanking period [23]. On the Atari 2600, out of the total 76 machine cycles it takes to display one scanline, 54 of these are for displaying the graphics, and the remaining 22 machine cycles are for the horizontal blank and is used to prepare the graphics for the next scanline (Figure 3). To display an entire screen, it will take 14,592 machine cycles.

$$\begin{aligned} DisplayTime &= 76mc/s \times 192s \\ &= 14592mc \end{aligned}$$

To create an asymmetrical environment, it requires precise timing to ‘race’ this beam, loading data into the playfield registers just before the beam reaches the right side of the screen. Doing this too early or too late can result in the picture becoming corrupted. As will be explained later in more depth, something to consider is that most 6502 instructions last three machine cycles instructions [23]. With the limited time, careful code design and skill needed, most developers prefer to mirror or repeat the playfield graphics [23]. Unlike these developers, however, this project opts to create an asymmetrical environment, and many of the quirks and consequences will be discussed later on in the development section.

2.3.4 Scanlines and Machine Cycles

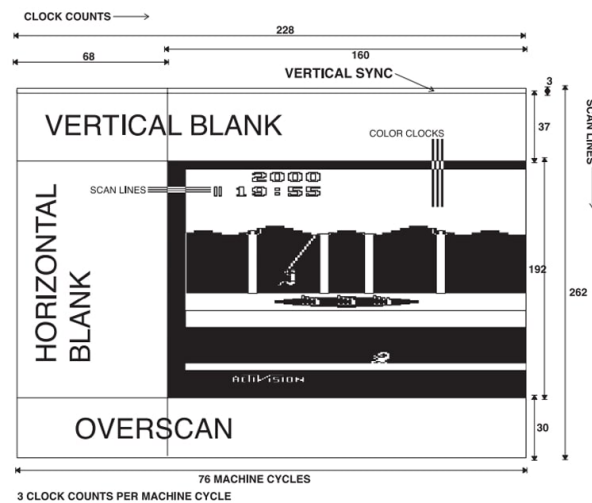


Figure 3: A diagram showcasing the vertical blank, horizontal blank and overscan periods, measured in scanlines in machine cycles [23]

Like the Horizontal Blank, the Vertical Blank is the period where the electron gun will return to the top of the screen after reaching the bottom [39]. Taking 2,812 machine cycles, this time is used for more complex computations and game logic [23]. The overscan period, lasting 2,280 machine cycles, is a slightly shorter period, and is to account for varying sizes of CRT TV screens at the time. In total, this means there are 5092 machine cycles to compute game logic before the electron beam begins displaying the TV picture. Therefore, the time to handle input or perform algorithms is very limited per frame. Of course, it could be possible to perform calculations during the picture being displayed, but this depends on the kernel (the code handling displaying the picture [18]).

Period	Scanlines	Machine Cycles
Vertical Blank	37	2812
Overscan Period	30	2280
Total	67	5092

Table 1: The number of scanlines and machine cycles non-display periods last.

While the 6507 contains a register called X and another called Y, these are not used for on screen coordinates and are instead general-purpose registers [39]. Carefully counting the scanlines and machine cycles instead is necessary for graphics on the screen [23]. Positioning sprites or playfield graphics vertically requires scanlines to be counted, which may be handled by one of the general purpose registers or perhaps value in memory. The only constraint about that is the limited RAM available. Positioning a sprite horizontally is more complex, and must be done through careful coding by counting machine cycles, strobing the RESPx address to reset the sprite to wherever the electron beam is at that moment, finely adjusting the position using the horizontal move registers [39]. Thankfully, reusable algorithms have been developed to handle this at the cost of scanlines and machine cycles [18].

How long these periods last and the resolution of the screen are influenced by different CRT TV schemes. These are the NTSC (United States, Japan, and others), which is described above, and the PAL (European countries, including the UK) and SECAM (Soviet Union, China, France and others) formats, which have a resolution of 160x228 [23]. Despite having more scanlines, these formats have different colour tables, totalling at 128, 108, and 8 colours respectively [39]. Even though PAL was used in the UK, NTSC was used for this project due to available documentation and wider colour palette. Modern emulators, such as Stella [35], can handle different formats without issue.

2.3.5 ROM Space

An additional challenge of the system is the amount ROM space for game code. Originally, cartridges came in either 2KB or 4KB (often referred to as simply xK, e.g. 2K) [23]. As technology advanced, ROMs with spaces with 8K or more became possible through a technique called bank switching, allowing the system to switch between two 4K addressable ROM spaces [18]. ‘Asteroids’ was the first Atari 2600 game to release using bank-switching [39], which allowed for more game code and sprite data. While absent in this project, these techniques have been used by the homebrew community to create new and better Atari 2600 games.

2.4 Power Wash Simulator



Figure 4: Box art from ‘Power Wash Simulator’ [30]

Released in 2022 and developed by FuturLab and Square Enix, ‘Power Wash Simulator’ [30] was chosen to be adapted due to its simplistic gameplay and artistic design in comparison to other modern games, suitable for this project’s scope. As can be expected, the game consists of various locations and vehicles for the player, solo or with friends, to clean by firing their power washer, blasting away dirt, technically being a first-person shooter mixed with a colouring book. After a section is cleaned, it will flash and a ding sound will play, creating an satisfying gameplay experience. Due to its simplicity, it has allowed some room to experiment with ideas to make the adaptation more unique without compromising the game’s identity, like what was done in ‘Halo 2600’ by Ed Fries [17], acting as the main source of inspiration for this project.

Being an adaptation of the sixth-generation console game Halo: Combat Evolved by Bungie [24], ‘Halo 2600’ was a successful, albeit limited translation of the original game, with recognisable elements but low resolution graphics. In the same way, this project, affectionately called ‘Power Wash Simulator 2600’, similarly does not create a one-to-one adaptation of ‘Power Wash Simulator’ as some features are impossible to translate, namely having bespoke levels with wholly unique items to clean. Consequently, additional inspiration has come from various other games, such as ‘Yars’ Revenge’ [20] and ‘Adventure’ [11], to explore the use of random numbers and procedural generation to navigate this issue, resulting in a partially unique game, one based on the source material but adapted to better suit the audience and capabilities of the Atari 2600.

3 Learning

3.1 Reading Books and Setting Goals

Before development could begin on this adaptation, the project began with the training phase, where the necessary skills in 6502 assembly and Atari 2600 game development were to be developed (although this was unexpectedly delayed until January due to other overwhelming University work). Serving as the primary learning material, ‘Making Games for the Atari 2600’ by Steven Hugg [18] goes through many examples of features that are possible on the machine, beginning with basic concepts about low-level programming, building then to moving the players, scoreboards, asymmetrical playfield graphics, procedural generation and even creating music.

In addition to that, the book also showcases the many challenges and caveats making a game for the limited system presents. For instance, while making code reusable is encouraged in modern video game programming [26], on the Atari 2600 it has costly consequences, adding additional machine cycles that perhaps some algorithms cannot afford. Admittedly, some of these drawbacks were also forgotten about during development, not necessarily out of ignorance, more so that there was so much technical detail to learn. Even so, learning all this took time, and even though time had already been lost, steps were taken to quickly get the project back on track.

To progress through at a steady pace, goals were set each day to, at this stage, study several related topics and chapters of the book in depth, organising knowledge so that it could easily be referred back to during development. Delving into the examples in the book, it granted a

better understanding of what could and could not be done, causing some parts of the original specification and possible desired features to be rethought (see Appendix E for a complete look at original ideas). With the benefit of hindsight, the original specification was too ambitious, and in some areas, unlikely to ever be achievable on the system.

With ideas now starting to form about more achievable features to implement, prototypes were made, exploring some of the examples in the book and being an opportunity to turn theory into practice, gaining some real experience using 6502 assembly. Despite a large amount of time trying to learn as much as possible, understanding 6502 assembly proved to be a major challenge as the basics of the language were hard to grasp. Indeed, it felt like learning how to program all over again. Not until much later in the project did it all start to make sense.

3.2 A Brief Introduction to 6502 Assembly

Assembly code is made up of human readable mnemonics, simple three letter instructions to the machine often with some operands, being only one step above machine code itself [18]. Using assembly instructions must be precise to do what is required, even creating simple loops can be challenging. Adding this, 6502 assembly code has to juggle data between the three general purpose registers, X, Y and A (the accumulator), using instructions like TXA (transfer X to A) or TAY (transfer A to Y). While these are "general purpose", even their use have limitations.

Being where most data is processed, instructions such as AND, EOR, ADC, SBC, take their operand and apply their operation to the value stored in the accumulator only, and cannot be used with the X and Y registers directly. Conversely, the accumulator cannot be used to index memory, whereas X and Y can, which can be used in combination with instructions like DEY (decrement Y) or INX (increment X) to navigate memory. Practically, this means that any memory address can be offset by the value contained in these registers. For example, STA TEMP, x stores the value at 'x' number of bytes away from the address TEMP into the accumulator, where x is the value contained in the X register (Y can be used similarly). In this way, code reusability can be introduced and it makes structuring data contiguously advantageous.

Subroutines can be defined within 6502 assembly and can be called similar to functions using the JSR instruction (for example JSR DrawPlayer). Control is passed back to the calling code by using the RTS instruction, similar to return. Doing so comes with a computational cost, where these instructions cost six machine cycles each, so any call to a subroutine costs at least 12 machine cycles plus whatever computation cost the subroutine does. When timing is not a concern, this is, in most cases, insignificant, but due to the TIA, where counting machine cycles requires precision and careful calculation, these few machine cycles become expensive.

Addressing mode	Machine Cycles	Explanation
LDA #2	2	Load literal value into A
LDA RESP0	3	Load value at memory address into A
LDA RESP0,x	4	Load value at indexed memory address into A
LDA (TEMP),x	6	Load value at indexed memory address pointed to by TEMP into A
LDA (TEMP,x)	6	Load value at memory address pointed to by indexed TEMP into A

Table 2: Example showing relevant addressing modes of the instruction LDA and the computational cost associated with it.

The same goes for the addressing mode of an instruction. As can be seen in Table 2, using a literal value, a memory address, an indexed memory address, or a pointer have varying costs [18]. Consequently, programming a game for the Atari 2600 makes the developer acutely aware of how these machine cycles are being used, meaning reusability needs to be considered carefully.

Working at a low-level, care must be taken about what is loaded into memory at all times, or how memory is used. The processor does not know anything about the data it is presented with unless the programmer says how to use it. As an example, the use of # tells the assembler that the data is to be used literally, such as ADC #2. This simply adds the value two to the contents of the accumulator. Subtly different, ADC 2 adds the contents of the memory location '2' to the accumulator, which may be any value.

Symbol	Base	Example
%	Binary	ADC #%00101101
\$	Hexadecimal	ADC #\$2D
None	Decimal	ADC #45

Table 3: Example of using different symbols to indicate different bases.

Additional considerations must be given to the type of data, either being decimal, hexadecimal or binary. Looking at Table 3, these are all valid ways of adding 45 to the accumulator, but it can be easy to forget or mix-up any of these symbols, leading to unintended effects that can be hard to debug (especially if, unluckily, the memory address '45' so happens to contain the value 45). With different formats of data flying around, it requires the developer to be considerate of what data is being used, how it is used and its computational cost.

3.3 Daily Logging

There are many nuances of 6502 assembly that will be further discussed in this document when they made their unfortunate appearances during development. Many of these are caught in "The Log", which is where all of the development notes were kept. During the project plan, agile methodologies were intended to be used, specifically drawing inspiration from Scrum, and the daily meetings involved [33]. For a solo developer, this was captured instead by daily reflections

and discussions after a day of development. Whilst also documenting development, these were key to figuring out solutions and problems.

Within this learning period specifically, "The Log" covered in detail different discussions: how each day of learning went; formulation of ideas; determining useful concepts, like random or generation, and graphics; defining scope and leaving out unimportant aspects, such as possibly audio and music; and most importantly goals for the next day, such as to read chapters 30-32. Starting this practice early would arguably be one of greatest successes of this project, setting achievable goals and showcasing the steady increase in proficiency in assembly programming throughout development.

4 Development

4.1 Tools

For development of this project, the 8bitworkshop IDE by Steven Hugg [4] was used almost exclusively to code in 6502 assembly, featuring an integrated emulator running in real time, showing the effects of changes to the code as were made. To aid development, it also includes many of the necessary files, mostly containing names for addresses and macros, syntax highlighting, debugging tools and GitHub integration, all that should be expected of a modern IDE, but was sorely lacking in 1977 [23]. Notably, the IDE includes more features like timing analysis that offer insight into how long certain sections of code are taking to execute, a tool used extensively throughout to tackle many of the issues the Atari 2600 and TIA chip present.

Additional tools created by the Homebrew community were used to create the graphics and audio for the game. Namely, Kirk Israel provides a web-based tool called 'Playerpal 2600' [3] to easily design player sprites, without needing to resort to graph paper unlike original Atari developers [23]. For audio, Kirk Israel has a similar tool called 'Sound Forger' [19] that was used to explore possible audio sounds for many of the aspects of the original game, such as washing dirt, firing the washer itself and that all important satisfying 'ding' whenever a section is complete. With these tools at the projects disposal, prototyping and designs for key parts of the game could begin.

4.2 Prototyping

Development began with prototyping some concepts, mainly two player movement, "cleaning" and most importantly drawing to the screen. These were intended mostly as an opportunity to develop proficiency in 6502 assembly programming. Of course it did so, but it was perhaps naive to think that these exercises would have been enough.

4.2.1 Cleaning

It was originally envisioned that the project would take gameplay inspiration from Yars' Revenge by Howard Scott Warshaw, one of the most popular original Atari 2600 games that pushed the bounds of Atari game programming [20]. Namely, some of the prominent gameplay aspects that

could be drawn upon for this project were the movement of the player, shooting horizontally and destroying the "Qotile's" shield.

In the game, the player can position themselves anywhere on the screen using the joystick, pressing the fire button to shoot a missile at the Qotile located on the right side of the screen. On contact with the Qotile's shield, which is made up of playfield graphics, only the part that was hit would be destroyed. With some simplification to make it achievable within the project time frame, these aspects could be translated to player movement and washing dirt in the adaptation, having that sense of familiarity for gamers old and new. Different objects, such as buildings, cars, and more could appear on the right side of the screen and the player could freely move to blast a constant "beam" or "stream" at the objects to wash them. After reading "Making games for the Atari 2600" [18], this constant beam is evidently impossible.

Horizontally, missile sprites appear to be between one and eight colour-clock across, depending on the NUSIZ register. For comparison, eight colour clocks are about two playfield graphics, much less than the almost 40 that may be required. One idea to perhaps navigate this was to combine both missiles and the ball together to form a longer beam, although even this would be limited to at most six playfield graphics. Making this solution further unattractive was the steadfast goal for the game to be two player, which would require the missiles to be independent, so the missiles cannot be combined. An alternative solution was to make the missiles fast enough that they would appear as beams, although this was rejected as it would cause flicker and make collisions harder to detect. The final nail in the coffin for this idea was the fact that missiles take on colour of the player on that scanline, meaning colour independency would be lost, and the beams may appear as not very water-like. This idea may have compromised both aims to adapt graphics and gameplay. Luckily, a different idea was presented.

Considering all these drawbacks, it was quickly changed to be a vertically appearing line, which should have been much simpler as missiles only need to be turned on or off per scanline, and can have independent colour, so long as the player is not also on that scanline. To prove this was possible, a prototype was built on top of one of the examples by Steven Hugg [18]. Entitled 'Async Playfield: Bricks', it is a basic but near wholly playable game similar to a classic Atari 2600 game called Breakout [9]. In it, the player can move left and right, and slightly up and down. A ball moves on its own across the screen, rebounding and changing direction when it hits a brick (the playfield graphics), gaining the player a point on collision. Considering that this project was going to be two player however, the ball could not be used as there is only one. Instead the missiles would need to be used, so all that was needed was to alter the code to display the missile sprite instead of the ball. Unfortunately, it was not this easy.

```
1      ; X register contains current scanline
2      lda #%000000000
3      cpx YBall
4      bcc .nomissile ; Originally BNE
5      lda #%000000010 ; for ENAM0 the 2nd bit is enable
6 .nomissile
7      sta ENAM0 ; Originally ENABL - enable ball
```

Listing 1: Altered code originally by Steven Hugg [18] to instead use the missile graphic.

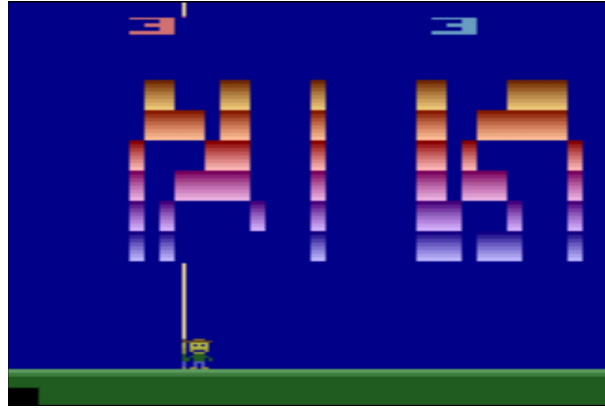


Figure 5: A screen capture of the prototype cleaning function built upon the original code by Steven Hugg [18]

Looking at Listing 1, this change was made by changing ENABL (enable ball) to ENAM0 (enable P0 missile), as well as changing BNE (Branch Not Equals [to zero]) to BCC (Branch [if] Carry [flag is] Clear) to achieve the "beam" effect that can be seen in Figure 5. BCC can be used similar to 'if (A < B)' for unsigned numbers [18], as can be seen in Listing 2, whereas BNE is similar to 'if (A != B)' (BEQ for the opposite). At first sight, however, the current use of BCC been used incorrectly as the missile appears constantly by the player and where the scoreboard is, and never above the player and in the bricks area, where it needs to be. Understandably this lead to confusion at the time, believing that the use of branch statements was incorrect, possibly needing BCS instead (essentially the opposite). Despite that, it showcased the possibility of using the beam to blast away the bricks, forming the basis for the cleaning. With more knowledge now, this experimental code is indeed correct and was not the cause of the problem. Nevertheless, it meant a new solution needed to be found to draw the beam properly, requiring skills that were yet to be developed.

```

1      if (YBall < CurrentScanline) {
2          DisableMissile();
3      } else {
4          EnableMissile();
5      }

```

Listing 2: Abstraction of Listing 1

4.2.2 Two Player Movement

Comparatively, adapting existing code to have two player movement was much easier and successful. Using the same example from the book, all that was needed was a duplicate of the first player (P0 from now on) movement subroutine for the second player (P1), only changing the bits that were read from the Joystick. According to the Stella Programmer's Guide [39], the SWCHA address stores all the joystick directional input for both players in one byte, where the higher and lower nybbles correspond to P0 and P1 respectively. Simply, the Joystick is purely digital and each of the four bits correspond to a direction.

Looking only at horizontal movement in Listing 3 of the movement subroutine (vertical movement was similar), the horizontal position of P0, XPos0, is loaded into the X register so it can be easily decremented or incremented, moving left or right respectively, while the accumulator only needs to deal with checking which direction is being input. Only once each direction has been checked will the new value be stored in XPos0 again. An alternative choice, and one more familiar in modern languages, may be to directly change XPos0, but this would, firstly, cost more machine cycles (DEX only costs two machine cycles, whereas DEC XPos0 costs three machine cycles), and secondly it would need data to be juggled around so it can be compared properly to the maximum values for the player position.

Speaking of, the player can only be at a maximum position of 153, the far right of the screen, or at value zero. Due to the screen having a horizontal resolution of 160 and that a player sprite being eight bits wide, the player perfectly fits to the remaining length of the screen. Likewise, and more obviously, the player is stopped whenever they reach the left side of the screen at position zero. Interestingly, to accomplish this, a comparison must be made with #1 not #0 due to the comparison BCC being equivalent to $A < B$, and not $A \leq B$. At the time, this difference was not wholly understood, resulting in an overflow error where the player's horizontal position would become 255, preventing right movement and causing glitchy displayed movement as erroneous data was being passed to the algorithm that actually set the position of the sprite.

As discussed, machine cycles need to be counted to position the sprites horizontally on the screen, and there is no concept of x-coordinates on the Atari 2600 outside of whatever abstraction in memory the developer has made. Fortunately, Steven Hugg presents a reusable algorithm to calculate where to position each sprite called SetHorizPos [18] (see Listing 10). At the cost of at least one scanline and made reusable due to the RESPx (coarse positioning) and HMP0x (fine positioning) addresses being contiguous in memory for the five sprites, the algorithm handles the actual counting of machine cycles and strobing the appropriate reset address, without needing the developer to do it themselves, offering some level of abstraction. With more objects, however, more scanlines are lost, exposing a limitation of the subroutine.

After all that, two player movement was a simple matter of duplicating the movement for P0 for P1, changing the bits checked according to the Stella Programmer's Guide [39], and calling the SetHorizPos subroutine for both P0 and P1 in the Vertical Blank, loading the registers appropriately, using a layer of abstraction with the use of the variables XPos0 and XPos1.

4.2.3 Scoring

Within his book, Steven Hugg presents an implementation of a scoreboard, utilising unique hardware functionality of the TIA [18]. Much like how the playfield graphics can be set to be mirrored or repeated, setting the second bit of the CTRLPF register will enable 'score-mode', where the playfield graphics will take on the colour of P0 and P1 on the left and right hand sides of the screen respectively [39]. Alongside this, the 6502's Binary Encoded Decimal (BCD) mode is used to calculate and display each player's score [18].

BCD allows the 6502 to work with decimal numbers up to 99 by splitting a byte into two to represent the two digits of a decimal number. As an example #81 is a hexadecimal number, representing 129, but in BCD it represents the decimal 81. By using SED to set the D flag,

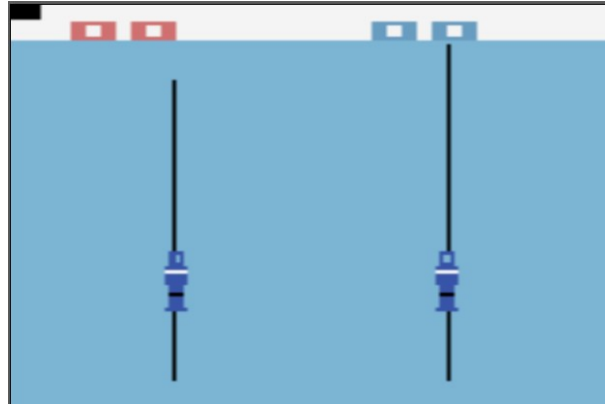


Figure 6: A prototype version of the players being able to move and shoot with visual glitches.

instructions such as ADC (addition) and SBC (subtraction) can be used for decimal arithmetic and not just binary. Unfortunately, this does not apply to instructions DEC and INC, which would continue to use the values as a literal binary value [18].

Using a lookup table for the font to be displayed, the final part of this is to convert the BCD values to numbers on the screen to make the scoreboard. In the final product, this display routine remains unchanged from the book, other than the colours. On the other hand, the clear limitation of this is that three digit scores cannot be displayed without more memory use. However, this was acceptable as these scores were going to display the percentage score of the dirt cleaned, a recognisable UI feature from the original game, but that calculation was far beyond the skill level at this point. At this time, it was only incremented every time P0 pressed the button on the joystick and fired their washer, showcasing that the scoreboard would work.

4.2.4 Shooting

To display the player anywhere on the screen and experiment with shooting, a basic kernel was implemented. A kernel within Atari 2600 game development often refers to the code that is responsible for drawing to the display, mostly comprising of loops to execute the same code for large portions of the screen [18]. Even though it is the period with the most scanlines, there is more concern here about falling within the limited 76 machine cycles per scanline, otherwise flickering may be introduced by subsequent scanlines being off by only a few machine cycles. At this time it was not a concern, so each loop of the kernel simply called the subroutine to draw the player. However, this subroutine requires two scanlines to execute, making it a two-line-kernel, allowing for more time at the consequence of a lower resolution [18]. Combining this with movement, the player was now able to move wherever they desired.

After combining the movement and scoreboard, it was time to finally incorporate shooting. Such a "simple" feature came as a shock as it was way more difficult to do than expected, mostly due to a poor understanding of assembly. Taking days to implement, it first began with a simple implementation that displayed the missile on every scanline and followed the player. Eventually, it was updated so that it would fire in an upwards direction from the player, wherever they were on the screen, which was desirable. However, moving the player would cause the missile to continue going and wrap around, likely due to an overflow error. Additionally, due to the code

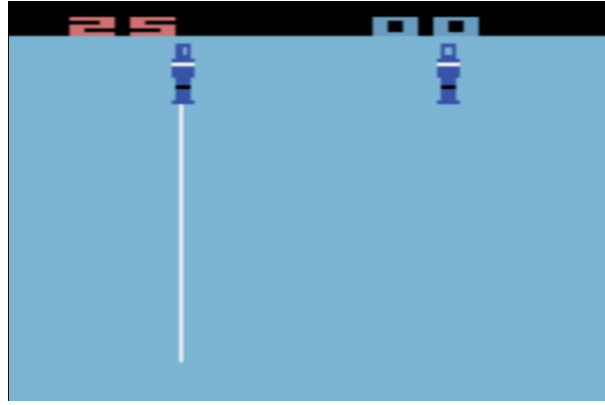


Figure 7: The prototype build presented at the Student Conference, showing player movement, scoreboard and firing

to draw the player and the missiles being coupled together, moving the players would cause the missile to disappear along the scanline where the other player was. Furthermore, another issue appeared where if the players moved to the other side of the screen, the movement would be very choppy. After more experience, this was likely due to memory not being correctly zeroed out before calculations were done, but was this hard to understand at the time and the issues were overwhelming; every minor improvement felt like a massive success.

After more experimentation, it was found that firing downwards instead of upwards performed the most correctly, although moving would cause it to wrap around to the top of the screen still. To additionally avoid an issue where the player sprites would appear stretched when they moved towards the bottom of the screen, it was decided that the players will be restricted to the top of the screen, only able to move left and right, losing total control of the player position. While not being the original vision, it was a compromise that felt worthy to present at the student conference, although it felt like substantial work for unimpressive results so far. With the upcoming conference, development halted and focus shifted to the presentation before development would continue.

4.3 Becoming Lost in Code and Doubts

Undoubtedly, the progress displayed here was disappointing at the time, plagued by a poor understanding of assembly. Looking back now, with more knowledge, its easy to be critical of the progress and see where issues could, and did, arise from, such as BCD values not wholly being understood or non-zeroed out data. On the other hand, this was precisely what this time was meant to be, forming a deeper understanding of assembly in a practical sense, and, crucially, making mistakes to learn from. Even so, assembly programming was daunting and doubts were beginning to be felt about whether this project was possible, perhaps too ambitious. To avoid burnout, and maybe out of fear, a break was much needed. Arguably, work could have continued much sooner after what became a two week break (granted other University work needed to be done too), and with this at the back of the mind, development saw accelerated progress.

After some time away from the project, time was spent getting reacquainted with it, trimming un-

necessary parts, making optimisations and adding very necessary comments. These comments were more so markers because it was so easy to get lost in the code, becoming an increasing problem as the code became much longer. It all looked very similar, and trying to understand the code was harder. Consequently, frequent comments seemed necessary, although perhaps much time was lost, wading through all the code, longing for the abstractions modern languages have.

Nevertheless, development began to pick up pace, beginning with small features to ease into assembly programming better than before. Up until now, due to the more experimental nature of the prototypes, only one player was able to fire their missile. This was expanded to both players, first by implementing the code to move the missile and then drawing the missiles to the screen, which took more time as the kernel needed to be remade to have enough time for the logic on each scanline. To more easily distinguish the players, the second player was recoloured to red (requiring a new colour bitmap), and the scoreboard colour altered to reflect this. More optimisations were made to other parts of the code and finally, after gently settling into assembly, it was time to draw playfield graphics and race the beam.

4.4 Playing with Playfield Graphics

4.4.1 Getting Dirty

In the original game, the players wash away dirt from recognisable buildings and vehicles. For example, the first level (or job in the context of the game) is cleaning a van. Later on, players wash houses, fire stations and even temples. All of these will be entirely covered in a layer of dirt, some parts being a bit stronger than others. Washing this away will reveal eye catching colours and interesting details [30]. Essentially, the entire game can be imagined as some kind of reverse colouring book.

Adapting this to the Atari 2600 has to come with some compromises. It was always intended that playfield graphics, the least detailed object on the console, would be used as these are the largest objects and there are more of them. As consequence, the objects to clean can only be at most 40 "pixels" across, losing the detail. Steven Hugg's "Bricks" example again was used as inspiration for how these playfield graphics could disappear from the screen dynamically [18], although this requires a lot of RAM.

There are three playfield registers on the TIA; PF0, PF1 and PF2 [39]. As a reminder, only PF0 is only four bits, whereas PF1 and PF2 are the full eight bits, totalling 20 bits. To have an asymmetrical display, the contents of these registers need to be carefully swapped out while the picture is being drawn, which was necessary for the game. Setting the CTRLPF register to repeated, the picture would be displayed in the following order; PF0-PF1-PF2-PF0-PF1-PF2 [39]. Therefore, to store what needs to be displayed, each row needs six bytes. Adding more rows can quickly increase the amount of RAM required, although initially this was set to six rows, totalling 36 bytes. While this took up 28.125% of memory, it still left plenty of room for other variables.

On the other hand, this does not leave any room for health values as described in the original specification. Realistically, this was probably never possible to begin with. One way this could have been done without limitations on memory is to have a byte for each dirt piece. For instance,

a piece of dirt can have a health value of one hundred, at maximum 255 for those tough pieces of dirt, and disappear only once it has hit zero. Using the 36 segments, this would have required 288 bytes of memory, 160 more than the 128 bytes available. Possibly, this could be halved so that a nybble represents a piece of dirt instead, cutting the maximum health to just 16. Even then, that would exceed the total RAM by 16 bytes. With this limitation in mind, its clear to see that health values were never likely possible for individual segments of dirt. It could be that the health could correspond to more than one segment, but then that likely needlessly overcomplicates the already defined six bytes per row that correspond to six segments.

Related to this, the original specification stated that the health should be communicated through changing colours. Again, that was never likely to be possible as playfield graphics can only be the same colour per scanline [18]. Changing the contents of the COLUPF register to change the colour mid-scanline would require the same careful timing as the already time critical asymmetrical display code. Even then, that is only for each segment, changing for each bit is probably impossible. As mentioned, the time it takes for one instruction to be executed, three playfield graphics would be drawn. Therefore, precisely colouring a playfield graphic cannot happen as there is not enough time, disallowing the changing colour for health, as well as the possibility to reveal an image underneath the dirt other than the background colour.

Consequently, it was decided that the dirt will make up the image of the object that would be displayed, and a piece of the image will disappear when the players' missiles collide with it (later feedback suggested that this looked more like destruction than cleaning). However, to do this, first the image needs to be drawn to the screen.

4.4.2 Drawing Dirt



Figure 8: A screenshot of the asymmetrical playfield graphics used for the dirt. Missiles are absent.

To create an asymmetrical environment, the electronic beam in the CRT TV needs to be raced, with each instruction being carefully timed. Listing 4 is only one part of the main kernel, specifically dealing with drawing the dirt to the screen in the final game. Based on code by Steven Hugg [18], this code went under several iterations to solve issues as new features were added. What it does is fairly simple. Contained within the larger DrawLoop, which keeps track of the row the

code is currently on by incrementing the Y register, the inner loop loads the dirt into the playfield registers based on the current row, looping until the value in the `DirtHeightCounter` variable reaches zero, giving each row a height in scanlines. After this, the instruction `STA WSYNC` is used to wait and ensure that the next piece of code begins when the electron beam is back to the left side of the screen [39]. Only then the left hand side bytes of the dirt data are loaded into playfield registers.

All of the dirt is contained within a contiguous part of memory beginning at the address labelled `Dirt`. In modern programming languages, this area of memory is similar to a two dimensional array in a column-major format, where the first block of memory contains all the data for the first PF0 segment, the second block for PF1 and so on. Each block of data contains N bytes, where N is the number of rows, although there is no separation between each block. As such, code needs to be carefully made to ensure the next block is not read or written to incorrectly.

To access one of these blocks, the offset needs to be calculated, which is done by the assembler, allowing for '+' and '*' symbols to calculate the required memory address. For example, `LDA Dirt+NumDirtRows*3` accesses the fourth block of data and is replaced by the assembler with the correct address. To access a specific piece of data, this is done programmatically using the Y register to retrieve the value. In this case, Y contains the row the code is currently on. These are then loaded into the accumulator and then stored in the correct playfield register.

Importantly, each of the LDA instructions cost four machine cycles and the STA instructions three machine cycles. As a result, completely loading the three registers with data costs 21 machine cycles, just enough to fit in the HBLANK period, the period before the picture is drawn, which lasts 22 machine cycles. In total, loading the two halves of the screen costs 42 machine cycles, leaving 34 machine cycles for other instructions, including determining if the loop is finished and an INX instruction between the loading the left and right halves for timing the beam. Other than these though, this does not leave enough time for anything complex before the next scanline, such as displaying the missile.

4.4.3 Cleaning Dirt

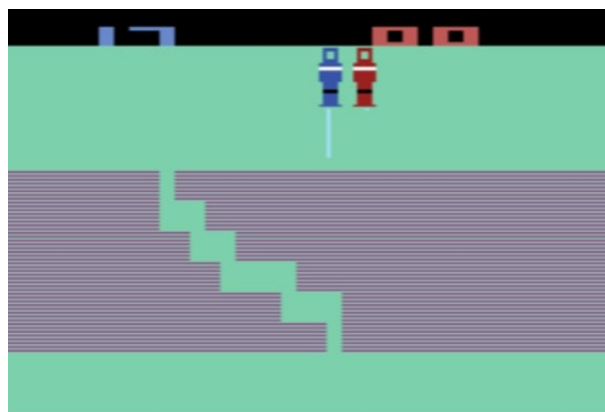


Figure 9: A screenshot showing the early dirt drawing and washing

One aspect left out while developing this was displaying the missile for both players. As this relied on the basic kernel in the prototypes, it was decided that it would be easier to develop the asymmetrical code without needing to worry about the missiles, hoping that the number of machine cycles left would be enough. However, only 21 machine cycles were left now and, while this may have been enough for one player, this was not enough for both. The only way to get more time was to use another two-line kernel.

Returning to Listing 4, code to display the missiles occurred between the two STA WSYNC, using the simple code covered in Listing 1. Consequently, while this doubled the amount of time available, the dirt now appears in a striped pattern rather than solid (missiles are unaffected), which was a reasonable compromise and was not too distracting. On the other hand, doing this caused an issue with the display, where the contents of the playfield graphics were still loaded in the playfield registers (see Figure 13). Neglecting to zero out memory like this highlights the importance of knowing what is loaded exactly where in such limited systems.

Once it was now possible for the missile and playfield graphics to overlap, collisions could be detected. Uniquely, the console hardware can detect collisions, not requiring any complicated software [23]. Using the CXM0FB register (CXM1FB or CXM0FB+1 for P1) [39], collisions can be checked between the playfield graphics and the missiles. Such a task is not time critical, so it is checked at the end of a frame. Using code originally by Steven Hugg [18], if the collision was detected, the area in memory for the dirt that was hit is cleared (Listing 9).

Implementing this was not without issue. Firstly, collisions were not zeroing the dirt data at all. At this point, just touching a playfield graphic increased the score counter for the player, so it was obvious that collisions were being detected, just that the CleanDirt subroutine was not working correctly. After some alteration, this was fixed but was now limited to the the bottom row of the dirt, which was odd considering that the top row of dirt was hit. Utilising the debugger in the IDE [4], and after much time and difficulty, it revealed that the code calculated the collisions to be on a much higher row number than it actually was. Once this was fixed, the player was now able to wash away the dirt and the game started to take shape.

4.5 Aiming

4.5.1 Intuitive Controls

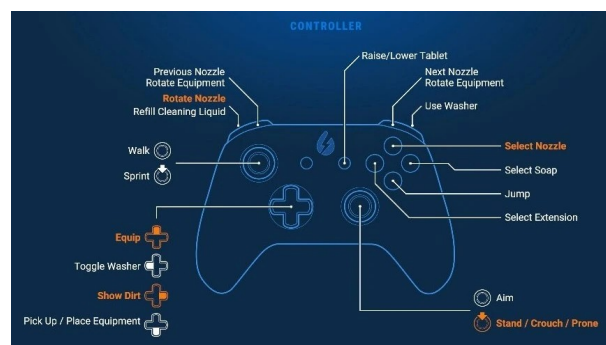


Figure 10: A screen capture of the control scheme for a gamepad controller in Power Wash Simulator[38].

After some early play tests, it revealed that the controls did not feel right. Firing the power washer at this point would make the beam slowly travel down the screen, uncontrolled by the player (other than being on or off), until it reached the bottom, washing away any bits of dirt it encountered. Moving the player while the beam was on would only wash away dirt on the row the beam was currently on, making it hard to hit dirt located in the middle, failing one of the objectives of this project to make the controls intuitive. To fix this issue, one idea was to make the beam return to the row closest to the player, but this did not feel intuitive to control either. Requiring a rethink, inspiration came from the controls found in the original game.

Modern first person shooters, including Power Wash Simulator, benefit from the innovations of software and hardware across several decades of gaming. In 2001, Halo: Combat Evolved pioneered the dual-stick control layout seen in most first person shooters since then [24], where the left stick is used for movement, and the right stick is used for aiming. In its own adaptation to the Atari 2600, Ed Fries noticeably leaves the aiming portion of this innovation out, making any form of aiming dependent on the player position, which is not unreasonable considering there is only one Joystick [17]. As a result, the game more closely resembles classic games like Adventure [23], rather than any modern day controls.

From this though came an idea to do the opposite, prioritising aiming, allowing the player to control where the beam is rather than their character. Fortunately, ever since the change in the prototype, going up and down on the joystick did nothing, meaning no other gameplay function was sacrificed. Returning to the original implementation, adding vertical movement was no different to horizontal movement other than changing a new variable for the location, and limiting the maximum value downwards to be the last row of dirt, which is calculated by the assembler. Ultimately this lead to the final implementation of the controls, as can be seen in Listing 8

Testing this, it felt much more natural to control. However, it also revealed a new problem where is was now hard to know where the beam was vertically, as it could still be moved when it was turned off. What was needed was some kind of reticle to show where the missile would be before it was fired, replicating the reticle found in the original game.

4.5.2 Reticles and Bugs

Initially, implementing a reticle appeared simple to do. As there was already the code to display the missile, the missile sprite could be reused when the player was not holding down their button, showing the point where missile is only and not the complete beam. Therefore, this seemed to be a matter of only adding another branch statement, BEQ, to turn on the sprite when the scanline was equal to the missile's vertical position.

Surprisingly, adding this caused an issue with movement, making the player jump from place to place, as well as the missile becoming out of sync with the player's position. At the time, it was peculiar and hard to reason why this was happening. After some time and debugging, it was revealed that a BIT instruction (similar to AND but only sets the flags [18]) was incorrectly comparing data to whatever was left in the accumulator during program execution, altering different memory locations that must have related to the player movement. Fixing this issue only required loading the right data into the accumulator, adding machine cycles that would cause another issue later on.

	Machine Cycles	Code
Original Code	7	<pre>lda #%1000000 bit MissileVel0 bmi .DrawReticle0</pre>
Fixed Code	5	<pre>lda MissileVel0 bmi .DrawReticleVel0</pre>

Table 4: Comparison of code solutions to save machine cycles

Once this first issue was resolved, reticles could be fully implemented, which turned out to be a bit more complex. First of all, to ensure that dirt was not washed away while in reticle mode, input data needed to be read and saved into memory before the frame began, and the code to handle collisions needed to be skipped. Becoming more proficient in assembly programming, this type of task started to become easier to implement. Now, a reticle was being displayed for both players and it was reasoned that increasing the size of the missile sprite and changing its colour to match the players would make it easier to see and more distinguishable.

Making use of the NUSIZx registers (see Listing 15), which handle the number and size of sprites, the missile width was doubled to two colour clocks (experiments with four and eight colour clocks width looked too large) [39]. Vertically increasing the size was not considered. Doing that may have cost more machine cycles, a finite resource per scanline, which turned out to already be exceeded. Nevertheless, the reticle was now clear on the screen for both players and thorough testing of it began.

During testing, both players were tested firing their washers. However, this took the longest path through the code, needing 77 machine cycles, a mere one cycle over. Even this small overshoot had a dire cost, resulting in the picture flickering as all the subsequent scanlines were off by some amount of time. Being an eye-sore and possibly dangerous to some players, it was imperative that this needed to be solved, whilst maintaining the reticle feature.

Through technical knowledge of the hardware, the number of machine cycles was cut by four machine cycles (code is repeated for both players) by removing an unnecessary BIT comparison for each player reticle. The `MissileVel0` variable contains the exact bit values for the button input per frame; if bit seven is set, then the button is not pressed [39]. Using two's-complement, this indicates a minus value that can be checked with the BMI (Branch [if value is] Minus), directing the program to draw the reticle, otherwise continuing to draw the full beam. As a result, both beams and reticles could be drawn with no further issue, improving the intuitiveness of the controls.

Looking back, it is a clear oversight but a costly one, not only in the number of machine cycles, but also the amount of time it took to spot. Much analysis went into resolving this issue, counting machine cycles through each path to determine if it was even possible to save a few machine cycles. If it was not possible, it could have meant more time would have been needed to devise a new solution, or even cut the feature entirely. Luckily, an obvious mistake made this straight-

forward to resolve, unlike another different issue.

4.5.3 Going Beyond the Beam

Based on informal feedback, the players were moved from the top of the screen to the bottom of the screen instead. Initially apprehensive to do so because too much code assumed the players were firing downwards, changes to the code were much easier than first thought, owing to an increase in skill in assembly. Whilst the basic functionality survived this transition, displaying the playfield graphics did not. A new visual bug was introduced where the last three bits of dirt on the left hand side were disappearing when the player washed away parts on the right hand side, suggesting the timing was off (see Figure 16).

To solve this, combinations of inconsequential two or three machine cycle instructions were added, at best solving the issue up to the last bit. Any more would cause further issues, including one completely unexpected. As a reminder, all of this code, the main kernel, uses branch statements to contain all of it within one loop, becoming longer with increased code complexity. It so happens that the code was just edging on the point of the maximum jump distance branch statements can take. Perhaps overlooked during the training phase, branch statements can only jump to memory locations no more than -128 or 127 bytes away [18]. Adding too many instructions resulted in an error message from the assembler, saying the jump was 131 bytes away, as can be seen in Figure 14. Therefore, the loop needed to be restructured rather than new parts added.

Sitting there the whole time, a five machine cycle instruction, `DEC DirtHeightCounter`, was promising due to its odd and large cycle count. Originally coming after the second PF2 data was loaded into the register, a place where its cycle count did not matter so much, it was moved to before the first PF0 register value was read. Consequently, this pushed out the machine cycles for the loading the first PF2 out of the `HBLANK`, but that did not matter. At that point PF0 was being drawn, and by the time PF2 was reached, the data would be safely loaded before PF2 was loaded again, solving the issue of the bits being overwritten before drawn.

Such a solution highlights the unique issues the Atari 2600 presents, and can only be solved with sufficient knowledge about the operation of the machine and the code. While interesting, it came at a terrible price of time. Developing anything in assembly is more time consuming, debugging even worse. Despite this, the core game was nearly complete. After increasing the amount of dirt from 36 to 72 to have more to clean (worsening memory consumption to 56.25% of RAM), and decreasing the dirt height to make each segment more square shape, it was time to make this supposed game more game-like.

4.6 Making a Game

4.6.1 Winning

With the game at a more presentable level, more people could now test the game out in an informal capacity, with their feedback crucially informing the next steps. While generally being positive and fun, people wanted to know how do you win and what happens when you do. In the original game, "winning" means completely cleaning the level of dirt, which may then unlock

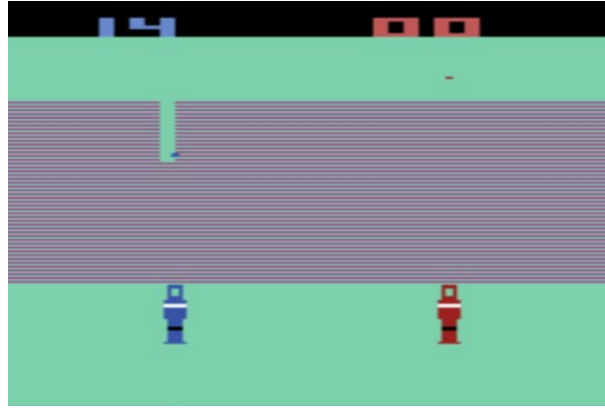


Figure 11: A screenshot of the completed drawing, with reticles and 72 dirt segments.

another location for the player to clean in the menu [30]. To begin adapting this, the game first needed to know when the dirt was completely cleaned.

With machine cycles being in abundance in the overscan period, a solution was devised to loop through the `Dirt` memory and test if it was all cleared, without needing to incur any further memory cost. At any point it was not, this signified the current game should continue. Clearly showing improved understanding of assembly, the algorithm almost worked on the first iteration, although it neglected to consider that only the higher four bits of `PF0` are set. By masking out the lower bits, the algorithm was fixed and now functioned correctly. As can be seen in Listing 9, to indicate if any memory is not cleared, the subroutine "returns" a negative value in the accumulator, or otherwise zero, at which point the program will reload the dirt memory, allowing the game to go on forever.

Once now there was a way to test when a game was over, the scores could be compared against each other to determine the winner. To indicate this, a win screen was used by slowly flashing the winning player's colour about three-to-four times over three seconds, using a counter set to 180 and knowing that the NTSC standard operates at 60Hz [39], replicating old Atari games. Using text was out of the question, requiring a lot more memory use to store the font for the letters and an entirely new kernel to be made. Colours were easier to implement and communicate who won to more users, such as those who speak other languages.

Despite all this, a major flaw with this system was that the score was increased for either player every time they washed away a piece of dirt. With well over 100 pieces of dirt to be cleaned, players getting a score more than 99 would cause an overflow error and their score reset back to zero, resulting in the game possibly incorrectly indicating the other player had won. However, this was all temporary, and it was intended to be replaced with a percentage value.

4.6.2 Calculating Percentages

In the original game, progress is indicated on the UI with a percentage counter in the top left corner, slowly increasing as dirt is blasted away. Being an iconic aspect of the original, it was always desired that this be adapted. What makes this challenging is that 6502 assembly does not have any multiply or divide instructions, nor work very easily with 16 bit values [18].

Algorithm Part	Machine Cycle Cost
Multiplication by 100	approx. 140
Division	53 per iteration
Update Display	up to 600

Table 5: Machine cycle cost of each part of algorithm to calculate and display percentage.

To avoid complex code, one idea was to limit the number of dirt bits to some known value, such as 200, or more easily 100. Consequently, even though it was simpler, this would not grant the freedom to make any detailed or recognisable aspects from the original game, compromising one of the aims of the project. Furthermore, with the intention that objects would be randomly generated in some way, it would be harder to ensure that only a certain amount of bits were generated without impacting graphical or gameplay quality, whereas the current possible 440 bits was more reasonable, at the cost of more memory and computation

In modern day programming languages, calculating percentages would be trivial to do, simply `float result = value \ total * 100`, possibly giving a decimal result. With realistically only at most 16 bits to store this, this would be much harder to represent on the Atari 2600, although it was thought that by first multiplying the value by 100 then dividing, the result could be rounded, avoiding the need for floating point numbers. On top of this, the BCD score would need to be updated to reflect this.

Using algorithms by Neil Parker originally for the Apple II [27], calculating the percentage had three parts: multiplication by 100 (using a series of left shift operations (LSR) and more complex addition), division by varying numbers, and updating the display. As can be seen in Table 5, each part was costly, especially updating the score, albeit naively implemented, detrimentally causing the picture to flicker even worse than before. Considering how many cycles are available in the underscan period, the multiplication part alone took up nearly 10% of execution for both players alone. While these algorithms may be suitable to the Apple II, it was evident they were not meant for the Atari 2600.

To remedy this, an idea was to split the work over three frames, one for each part. Successfully, this solution resolved the issue, but only revealed more faults. First of all, it was overlooked that the value on the scoreboard was in BCD, resulting in the values for the multiplication part being off. As an example, lowering the multiplication value to just five during debugging revealed that going from a score of 15 to 20 unexpectedly displayed 14 (20 in binary is the same as 14 in BCD). Further work resolved this too and subsequently another issue was discovered with the division part where only a value of 65 was displayed, or, if lucky, 64. More work, more time, something that was depleting with approaching deadlines. With much regret, it was time to admit defeat and divert attention to more achievable features, although not before a different solution was found (the cut code for percentage calculation can be seen in Listing 17).

In the original game, a satisfying 'ding' and visual effect played whenever a section is complete. Coincidentally, the concept of sections or segments was already captured by the bytes that store the dirt data. A simple BEQ statement to add a score whenever a whole section had been cleaned was easily possible and is now included in the final game (Listing 6), adding a certain degree of strategy and competitiveness that the original game lacks too. Only when a last bit of a section

PF0	PF1	PF2
1111—	000000 11	11110000

Table 6: An example of how data may be split over three registers for two images.

is cleared will score be added, meaning that the other player can snatch up points to outdo their opponent. Such a feature was desirable, and perfectly captured how adaptation allowed for some changes too. Consequently, while one feature was lost, another was expanded upon to make this adaption more unique and fun. Even though it still is disappointing that the percentage counter could never be wholly translated, procedural generation was comparatively a resounded success.

4.6.3 Random Numbers and Procedural Generation

There are several ways to generate pseudo-random numbers on the Atari 2600, but sometimes the simplest solutions are more ingenious [18]. Returning to Yars' Revenge [20], random generation can be achieved by using the game code itself, considering that it is all bits and bytes, appearing to be more or less random over the course of development. Combining this with procedural generation, this was a possible solution to having multiple levels in the game without the need for creating new levels and taking up precious ROM space, similar to the solutions in classic Atari games like Adventure and 'Pitfall!' [23].

To begin with, random number generation was first implemented, creating a random pattern of dirt-bits on the screen, rather than the full generation where all the bits would be set thus far (see Figure 51). However, based on the informal play-testing sessions, people seemed to enjoy using this "blank" canvas to draw pictures, such as smiley faces. In fact, this is possible in the original game, even featuring in some of the promotional material [30]. With the clear appeal of this mode, it was kept in the final game and can be accessed by flipping some of the switches on the game console itself. With now a "blank" canvas and random mode being implemented without too much issue, procedural generation could be worked on.

One of the challenges initially was deciding how detailed the objects that the procedural generation would use should be. With only 40 playfield graphics for each of the 12 rows, the resolution was fairly limited to begin with. Splitting a byte into four so two bits represented a 10 by 12 bit image seemed to be a reasonable resolution with enough for four unique images (e.g. 10010011, could be split into 10-01-00-11, representing images 2-1-0-3).

Quickly, it became apparent splitting the playfield sections into four would not be so easy. Not only would the 10 bits need to be split across at least two bytes, the PF0 segment can only store four bits, thus meaning a complete PF0-PF1-PF2 section would be unevenly distributed (Table 6 shows an example of this). After some time, a naive implementation revealed an overlooked aspect: the PF1 bits are reversed [39]. To avoid the need to have lookup table for reversing the bytes taking up more ROM, or a complex algorithm, taking up dwindling machine cycles, it was simpler to increase the number of sections to six, one for each playfield register, at the cost of reducing the resolution.

Even then, this solution turned out to still be too expensive to execute on one frame, depending



Figure 12: A screenshot of the final completed game in play

on the image being loaded. Causing yet another flickering issue, lessons learned from the percentage calculation implementation meant that the answer was obvious, split the loading over two frames, first loading the left then right playfield registers (see Listing 12). As a result, the loading is smooth, at the consequence of lower resolution and fewer images, vaguely looking like various houses, a car, an office or a flag. Nevertheless a success, procedural generation was one of the last major pieces of functionality for the game, now all that was left was polish.

4.6.4 Animations, Sprites and Audio

Prior to procedural generation, as a reaction to feedback, the player sprites were updated. Inspired by *Commando* [11], a higher detailed bitmap was made showing the player facing the objects needing to be cleaned to increase immersion. To improve that further, a simple walking animation loop was included by changing between three frames depending on a frame counter value or if the player was idle (Listing 16). Below the players, graphics were further improved by adding a road of sorts to separate the players from the background more, although, given more time, the detail of this could be greatly improved.

Audio was never an important feature but desirable nevertheless, similar to how it does not have much emphasis in the original game, noticeably not having a soundtrack, only sound effects. Like anything on the console, the Atari 2600 is limited to only two audio channels capable of producing very *unique* sounds to say the least [18]. In order to adapt the sound effects in the original game, possible sounds were explored using Kirk Israel's sound forger online tool [19]. Despite the feeling of burnout and, admittedly, headaches induced by the frankly horrible noises the console makes (especially when incorrectly implemented), basic audio was finally included using code by Kyle (8Blit) [21] for washing, the satisfying ding from the original game, and a passable jingle when the game is won (as well as a very necessary mute button using the P0 expert mode switch). While these aspects could see further improvement, the game was undeniably finished with only a few aspects to optimise and polish up, after that it was time for the game to go gold.

4.7 Publishing a Game onto a Physical Medium

While mostly the Javatari emulator was used throughout development, one of the aims of this project was to make the game playable on the original hardware. From nearly the beginning of the project, prior to any development, an Atari 2600+ system [8], a modern day recreation of the original console, and a Harmony Cartridge [2], a cartridge that allows any game ROM to be stored on it and inserted into the hardware, were obtained. Having these from an early point was a major benefit into understanding what the actual hardware was like, what it was capable of, and importantly what it felt like to play on.

Throughout development, ever since the basic dirt display and collisions were implemented, several builds were downloaded and played on the console, generally following the phases of modern day game development cycles, from pre-alpha to beta to going gold [34]. Crucially, these identified issues with the aiming, visual style and general fun. Now with development over, the ROM image is available to be played on Harmony Cartridges or emulators anywhere. To validate this, tests have been carried out.

5 Testing

.While there do exist tools to automatically test 6502 assembly programs, such as that by Asai Yasuke [5], these are more suited to conventional 6502 systems, such as the Apple II. Consequently, manual testing has been carried out to ensure the correct workings of the program.

5.1 Functional Requirements

Movement and aiming have been tested in a number of ways. First of all, Table 7 and 8 assure that the controls move the player left and right correctly, as well as move the beam up and down, without either input interfering with the other (as both are read from SWCHA). Additional tests on the Atari 2600+ console using the Joystick find these are intuitive, being perhaps a little stiff and hard to get used to, although that is due to the hardware and no fault of the software. Familiarity and skill may mitigate this issue over time.

Tables 11 and 12 ensure that a reticle or a beam is present when the button is pressed or depressed respectively for both players, while Tables 13 and 14 clearly show that dirt can only be cleaned when the player presses the button and only if a dirt piece is present at the same location as where the beam hits. Furthermore, Table 45 shows that each column and row can successfully be cleared.

Clearing segments entirely correctly gives the player who cleared the last bit an increase in score, as is shown in Table 16. To communicate this to the player, a ding sound plays, as shown by Table 19. Meanwhile, Table 17 shows the program correctly only ends a game when all segments are clean, otherwise it continues. When the game ends, Table 18 shows that the game correctly displays which player won the game or if there was a draw through an animation.

When a new game begins, Table 20 shows that the game correctly loads either in full, randomly or procedurally respectively, depending on how the switches on the console are flipped. For

procedural generation, test suites 22 and 22 ensure that the program correctly gets data from bitmaps and loads them into the full 8-bit segments and 4-bit segments.

Finally, adding polish to the game, animations correctly play when the players move left or right, according to test suites 25 and 26. Whenever either player fires, test suite 27 ensures that audio plays, and similarly, test suite 28 shows that a jingle plays when a player wins.

5.2 Non-functional Requirements

Outlined in the original specification, and even though it adapted over time, there were several non-functional requirements for the game. Dealing with the limitations of the console, the final game confidently comes well within the 4K ROM limit, coming in at only 2,026 bytes ROM, which is line with early original Atari 2600 games [23]. In comparison, the RAM usage comes in at 117 bytes, which leaves enough space for any data pushed onto the stack. Additionally, testing showed that any flickering was absent entirely, or at least imperceptible. Regarding this as well choice of colours, further tests with a sizeable audience held in a formal capacity may reveal how well these appeal to people more widely. For this project, there was no time to do this and could be improved upon.

6 Reflection

6.1 The Process and Improvements

Before the project began, one risk that may have affected the development of the software was not having the sufficient skills to make a program in 6502 assembly. Certainly, doubts and a feeling of being overwhelmed were felt early on in the project, lasting for some time. Gradually, these skills developed. Not until about half way into March, a considerable time into the project, did these feelings all melt away and it all began to click together, accelerating development. Considering this, more time to learn would have been enormously beneficial, but this had to be balanced with other University work.

Turning to the method of learning, jumping into developing a game in an unfamiliar language and extremely limited computational environment is perhaps not the most sensible choice. Online resources, such as Easy 6502 [15], offer a safer and better paced environment to learn in, with emulators and examples that build upon each other. Comparatively, Steven Hugg's examples [18] take large, daunting leaps in the complexity. With more time, or starting early, confidence and skills could have been developed much earlier. Moreover, it was severely underestimated how difficult the basic features were to implement, and the opposite for the more "difficult" features, like procedural generation. Fortunately, development overcame this shortcoming, finishing on schedule, despite some features being removed.

Removal of some features were nevertheless disappointing; percentage calculation namely presented some mixed feelings. During development, cutting this lowered morale dramatically, contributed to burnout, and depleted development time. Despite that, it was one of the most pivotal learning experiences during development, leading to the successful implementation of procedural generation. Development for the system has shown that it is mostly about making comprom-

ises, the current scoring method being one such compromise. Given more time, this feature could be revisited and possibly implemented to some degree, after some optimisations to other systems to save machine cycles. However, realistically this is still an eight-bit computer, making these kind of calculations especially difficult with larger numbers.

In comparison, other features may be possible to implement to improve the game. Unlike dirt and health, which are likely impossible due to memory constraints (or otherwise the amount of dirt would need to be limited severely), nozzles and player interference are aspects of the original game that were considered but never implemented. Being of low importance anyway, as the difficulty became apparent, these were cut to make sure time was dedicated to more important aspects. Without a doubt, these may be added to the game. Nozzles may take the form of power ups, possibly making use of the unused ball sprite, adding more depth to the game experience. Interference is arguably in the game already through point stealing, but is more so a happy accident. Locking out movement or stunning the player in some way may add further player interaction (that being said, nothing stops the players from interfering with each other in the real world either).

Returning to procedural generation, while generally a reasonable implementation, additional improvement could be made to the detail and number of unique items/images, which should not be too demanding considering that the end result is only about 2KB, plenty of space for more bitmaps. Hindering that though, as well as graphics in general, this project lacked sufficient artistic skill, so in many cases the art could be improved where software limitation is not an issue. This could include, for example, more sprites to animations to have a more lively firing animation, adding further polish. In a similar way, audio could be improved dramatically to be more appealing generally, possibly making flipping the mute button at least a little bit less attractive. With various sources of inspiration, namely *Commando* [11] (impressive graphics and catchy music), the system is clearly capable of even more impressive things with more deft hands.

While many aspects would have benefitted from additional time, other University work is still a matter that has to be considered. Therefore, with realistically two-to-three months available, the level of work completed is arguably adequate. However, there may have been ways where time could have been saved. An aspect not considered at any point was using AI to generate code, admittedly more so out of stubbornness, pride and an earnest effort to learn all about 6502 assembly programming. Some experiments with AI tools after the fact have shown that they are capable of producing reasonably competent Atari 2600 game code, but without any prior knowledge, this may more likely have been a hindrance to the learning process, considering it was one of the aims of the project. Now with sufficient knowledge, the emerging use of AI may be an interesting way to more efficiently develop features, such as any of those discussed previously, alongside competent programming ability.

6.2 The Product

Regarding the overall adaptation, it could be argued many of the major systems are in place to reasonable standard: cleaning, firing, movement, aiming, and objects to clean, even adding new features such as able to be load games in different ways to navigate the limitations on memory. Graphically, it is mostly unimpressive, with low resolution and low detailed bitmaps and sprites,

but still recognisably deriving from Power Wash Simulator. Of course, the game is full of compromises such as this but, ultimately, it showcases that Atari 2600 can still live up to its original purposes even now in the ninth-generation, going beyond the sixth-generation adaptation of Halo 2600 [17]. On the other hand, in terms of innovation, it hardly pushes the boundaries. Most of the concepts are taken from Steven Hugg's book, adapting those to suit the needs for the adaptation. Being the inspiration for this project, Halo 2600 is similarly hardly boundary breaking, itself taking inspiration from existing Atari games, such as Adventure [17].

While games like 'Pac-Man 8K' [14] and many Activision games [23] truly push the console to its limits, featuring impressive graphics and tricks to maximise performance, this project exists in the same area where Halo 2600 exists, as a source of inspiration. Halo, being an older game, may be losing its cultural impact, whereas Power Wash Simulator is still more relevant. Perhaps pushing this game to a wider audience may inspire more developers to consider creating their own games for the system, or explore other hardware. In this wider context, it shows how old hardware, be it the Atari 2600 or not, can still be a viable platform for boundaries to be pushed and innovations made.

More personally, this project mainly existed as an exercise on limitations and learning assembly languages. Now, with a greater appreciation for those who were on the frontiers of the game industry, the skills and knowledge developed as consequence from this project can be taken to other sectors where proficiency in assembly languages may prove useful. For example, work in the emerging sector of AI has dabbled with the possibility of using assembly languages to make optimisations to algorithms, providing a substantial boost in performance [22]. Not only does this project exist to inspire new work, new skills have been developed to transfer anywhere, peeling back the layers of abstraction present in modern day languages to now go into making new software with practical knowledge of the underlying systems on a low-level.

7 Conclusion

Since even its release in 1977, the Atari 2600 has been home to adaptations of games from superior hardware as per its original design goals. Games like Battlezone [7] or Asteroids [23] showcase that the console was capable of adapting arcade machine games to reasonable accuracy at the time, while 'Halo 2600' proves that is still the case even for newer sixth generation games. This project went further to show that still continues even to ninth-generation console games with this adaption of 'Power Wash Simulator'. While some compromise is inevitable, future work could continue to push the power of the Atari 2600 into further console generations. Beyond the scope of just the Atari 2600, this project highlights the viability for older hardware to still be developed for. More than anything, developing software for limited hardware in a low-level environment is an enormously beneficial exercise for any software developer, removing the layers of abstraction in modern languages, allowing the developer to gain a better understanding of the hardware, including the limitations and freedoms they present. While innovation and boundary pushing technology opens up for new possibilities to be explored, this project has shown the other possibility that existing innovations perhaps have yet to be fully explored themselves and that by constraining ourselves to some degree future solutions may be found in the past.

References

- [1] 'Re: MOS 6507 Tech Spec's' - MARC. URL: <https://marc.info/?l=classiccmp&m=110668039327589&w=2>.
- [2] "Batari". *Harmony*. URL: <https://harmony.atariage.com/Site/Harmony.html>.
- [3] Alienbill.com [kirkjerk]. *PlayerPal V2.2A*. URL: <https://alienbill.com/2600/playerpalnext.html>.
- [4] *8bitworkshop*. URL: <https://8bitworkshop.com/>.
- [5] AsaiYusuke. *GitHub - AsaiYusuke/6502_test_executor: A cross-platform unit testing tool for MOS 6502 assembly*. URL: https://github.com/AsaiYusuke/6502_test_executor.
- [6] *Atari 2600 Specifications*. URL: <https://problemkaputt.de/2k6specs.htm>.
- [7] Atari Inc. *Atari 2600 Manuals (HTML) - Battlezone (Atari)*. 1983. URL: https://atariage.com/manual_html_page.php?SoftwareID=859.
- [8] Atari®. *Atari 2600+ - Official Atari Video Game Consoles*. URL: https://atari.com/products/atari-2600-plus?srsltid=AfmB0op9zYlaIsZG926gNXpRFtP_WLYUeDddVrhxxbRPUEsXyV0dJpH-.
- [9] Atari®. *Breakout*. URL: https://atari.com/pages/breakout?srsltid=AfmB0ooY7nJAFSC_Tcp-nlCQ0VHaux0kukrG6nDTyKuRP0eT8smxY7B1.
- [10] Atari®. *History*. URL: <https://atari.com/pages/history>.
- [11] *AtariAge - Atari 2600 Manuals (HTML) - Commando (Activision)*. URL: https://atariage.com/manual_html_page.php?SoftwareID=938.
- [12] *AtariAge - Have you played Atari today?* URL: <https://atariage.com/>.
- [13] *CVGA Disassembled | Second generation (1976-1984) · Online exhibits*. URL: <https://apps.lib.umich.edu/online-exhibits/exhibits/show/cvga-disassembled/gamegen2>.
- [14] DINTAR816. *New pacman for atari 2600*. 26th Aug. 2014. URL: <https://forums.atariage.com/topic/229152-new-pacman-for-atari-2600>.
- [15] *Easy 6502 by skilldrick*. URL: <https://skilldrick.github.io/easy6502/>.
- [16] Dave Farquhar. *The Atari 2600 CPU: the MOS 6507*. 19th Jan. 2025. URL: <https://dfarq.homeip.net/atari-2600-cpu/>.
- [17] Ed Fries and Albert. *Halo for the 2600 released at CGE! Download the game here!* 1st Aug. 2010. URL: <https://forums.atariage.com/topic/166916-halo-for-the-2600-released-at-cge-download-the-game-here/>.
- [18] Steven Hugg. *Making games for the Atari 2600*. Puzzling Plans LLC, 22nd Dec. 2016.
- [19] Kirk [kirkjerk] Israel. *atari sound forger*. URL: <https://alienbill.com/2600/atari-sound-forger/>.

- [20] John Kauderer. *The addictive and unique gameplay of Yars' Revenge*. 28th May 2024. URL: https://atari.com/blogs/newsroom/the-addictive-and-unique-gameplay-of-yars-revenge?srsltid=AfmB0orhFWJ0nNZ3blJZ1AIVIyu7zIt_rY-Uc7xlbniQunDUDPEt_-y.
- [21] Blit Kyle. *8Blit - ATARI 2600 VCS Game Programming Episode*. URL: <https://8blit.com/s03e02/>.
- [22] Liviu Octavian Maftciu-Scai, Victor Deian Balutoiu and Roxana Teodora Maftciu-Scai. "Improving the Performances of Machine Learning Algorithms by Using Assembly Language". In: *2024 26th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)* (16th Sept. 2024), pp. 251–255. DOI: [10.1109/synasc65383.2024.00049](https://doi.org/10.1109/synasc65383.2024.00049). URL: <https://doi.org/10.1109/synasc65383.2024.00049>.
- [23] Nick Montfort and Ian Bogost. *Racing the Beam. The Atari Video Computer System*. MIT Press, 25th Feb. 2020.
- [24] David George Morten. *Ways the Halo series has influenced gaming*. 17th Nov. 2023. URL: <https://gamerant.com/ways-halo-series-influenced-gaming/>.
- [25] Nicole Newton. *xbox series x vs ps5 specs explained: which is more powerful?* 2nd Jan. 2024. URL: <https://www.musicmagpie.co.uk/blog/2024/01/02/xbox-series-x-vs-ps5-specs-explained-which-is-more-powerful/>.
- [26] Robert Nystrom. *Game programming patterns*. 2nd Nov. 2014. URL: <https://ci.nii.ac.jp/ncid/BB18119423>.
- [27] "Neil Parker". *Multiplying and Dividing on the 6502*. URL: <https://llx.com/Neil/a2/mult.html>.
- [28] Dave Parrack. *How Long Will This Console Generation Last? Here's My Best Prediction*. 30th Aug. 2024. URL: <https://www.makeuseof.com/how-long-will-ninth-console-generation-last/>.
- [29] *Platform Totals - VGChartz*. URL: https://www.vgchartz.com/charts/platform_totals/Hardware.php/.
- [30] *PowerWash Simulator | SQUARE ENIX*. URL: <https://powerwash-simulator.square-enix-games.com/en-gb/>.
- [31] Brent D. Rosso. "Creativity and Constraints: Exploring the role of constraints in the creative processes of research and development teams". In: *Organization Studies* 35.4 (5th Mar. 2014), pp. 551–585. DOI: [10.1177/0170840613517600](https://doi.org/10.1177/0170840613517600). URL: <https://doi.org/10.1177/0170840613517600>.
- [32] Ieee Spectrum. "Chip Hall of Fame: MOS Technology 6502 Microprocessor". In: *IEEE Spectrum - History of Technology* (4th Dec. 2023). URL: <https://spectrum.ieee.org/chip-hall-of-fame-mos-technology-6502-microprocessor>.
- [33] Apoorva Srivastava, Sukriti Bhardwaj and Shipra Saraswat. "SCRUM model for agile methodology". In: *2017 International Conference on Computing, Communication and Automation (ICCCA)* (1st May 2017), pp. 864–869. DOI: [10.1109/ccaa.2017.8229928](https://doi.org/10.1109/ccaa.2017.8229928). URL: <https://doi.org/10.1109/ccaa.2017.8229928>.

- [34] Nadia Stefyn. *How video games are made : The game development process*. URL: <https://www.cgspectrum.com/blog/game-development-process>.
- [35] Stella: "A Multi-Platform Atari 2600 VCS emulator". URL: <https://stella-emu.github.io/downloads.html>.
- [36] The Strong National Museum of Play. *Video Game History Timeline - the strong National Museum of Play*. 5th Dec. 2022. URL: <https://www.museumofplay.org/video-game-history-timeline/>.
- [37] Ernie Tretkoff. *October 1958: Physicist Invents First Video Game*. 22nd Sept. 2008. URL: <https://www.aps.org/apsnews/2008/09/first-video-game>.
- [38] Contributors To PowerWash Simulator Wiki, Altradil and Felidae13. *Controls*. URL: <https://powerwash-simulator.fandom.com/wiki/Controls>.
- [39] Steve Wright and Darryl May. *Stella (Atari 2600) Programmer's Guide*. 1st July 1988.
- [40] Ziv. *The best game engines you should consider for 2024*. 25th Dec. 2024. URL: <https://www.incredibuild.com/blog/top-gaming-engines-you-should-consider>.

Appendices

Appendix A: Development Photos



Figure 13: A screenshot showing a display issue after changing the kernel to a two-line kernel.

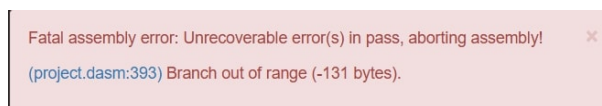


Figure 14: An error message from the IDE stating that the jump distance has exceeded 128 bytes

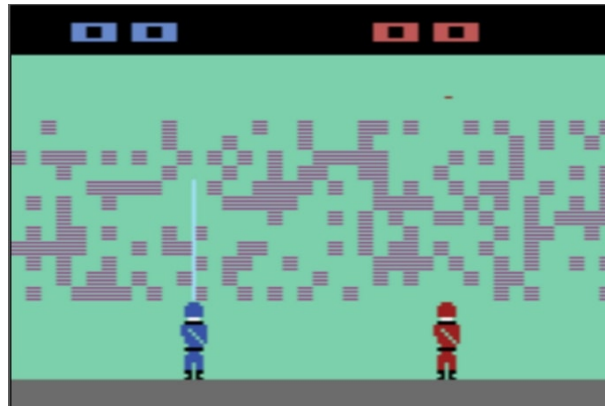


Figure 15: A screenshot of an early version of pseudo-random generation using the game code, eventually leading to procedural generation



Figure 16: A screenshot showing the issue where the last three bits on the left side were repeating the right side

Appendix B: Assembly Code

```
1 MoveJoystick0 subroutine                ; MoveJoystick1
2     ldx XPos0                          ; XPos1
3     lda #%01000000 ;Left?              ; #%00000100
4     bit SWCHA
5     bne .SkipMoveLeft
6     cpx #1
7     bcc .SkipMoveLeft
8     dex
9 .SkipMoveLeft
10    lda #%10000000 ;Right?              ; #%00001000
11    bit SWCHA
12    bne .SkipMoveRight
13    cpx #153
14    bcs .SkipMoveRight
15    inx
16 .SkipMoveRight
17    stx XPos0                          ; XPos1
```

Listing 3: Snippet of the MoveJoystick0 subroutine showing only horizontal movement. Subtle differences for P1 movement are shown in comments (e.g. ; comment).

```
1     ; Y contains the dirt row we are on.
2     ; X contains the scanline we are on.
3
4     ldy #$ff ; Start at -1 ;
5     ldx #$ff
6 DrawLoop
7     iny      ; Go to next brick row
8     lda #DirtHeight
9     sta DirtHeightCounter
10    cpy #NumDirtRows
11    bcs DoneDirtDraw
12 .innerLoop
13
14    inx      ; increment scanline counter
15    stx Temp
16    sta WSYNC
17
18    ...      ; Basic missile display exists here
19
20    sta WSYNC
21
22    ; HBLANK
23    lda Dirt+NumDirtRows*0,y
24    sta PF0
```

```

25     lda Dirt+NumDirtRows*1,y
26     sta PF1
27     lda Dirt+NumDirtRows*2,y
28     sta PF2
29     ; END HBLANK
30
31     inx          ; Increment scanline counter
32     nop          ; timing
33
34     lda Dirt+NumDirtRows*3,y ; 4
35     sta PF0          ; 3
36     lda Dirt+NumDirtRows*4,y
37     sta PF1
38     lda Dirt+NumDirtRows*5,y
39     sta PF2          ; 21 machine cycles
40
41     dec DirtHeightCounter
42
43     lda DirtHeightCounter
44     bne .innerLoop
45     beq DrawLoop
46
47 DoneDirtDraw
48     sta WSYNC
49
50     ...

```

Listing 4: Early version of the main kernel showing only code relevant to achieve asymmetrical playfield graphics, based on work by Steven Hugg [18]

```

1     ; Y contains the dirt row we are on.
2     ; X contains the scanline we are on.
3
4     ldy #$ff ; Start at -1 ;
5     ldx #$ff
6 DrawLoop
7     iny          ; Go to next brick row
8     lda #DirtHeight
9     sta DirtHeightCounter
10    cpy #NumDirtRows
11    bcs DoneDirtDraw
12 .innerLoop
13
14    ; Enable missiles
15    ; -----
16

```

```

17     inx      ; increment scanline counter
18     stx Temp
19     sta WSYNC
20
21     lda #0
22     sta PF0
23     sta PF1
24     sta PF2
25
26     lda MissileVel0
27     bmi .DrawRetic0
28
29     lda MissileY0
30     cmp Temp      ; If the scanline is more than the Missile Y,
                    ; disable the missile
31     bcc .EnableMissile0
32     jmp .DisableMissile0
33 .DrawRetic0
34     lda MissileY0
35     cmp Temp
36     beq .EnableMissile0
37 .DisableMissile0
38     lda #0
39     jmp .Finish0
40 .EnableMissile0
41     lda #2
42 .Finish0
43     sta ENAM0
44     lda MissileVel1
45     bmi .DrawRetic1
46
47     lda MissileY1
48     cmp Temp      ; If the scanline is more than the Missile Y,
                    ; disable the missile
49     bcc .EnableMissile1
50     jmp .DisableMissile1
51 .DrawRetic1
52     lda MissileY1
53     cmp Temp
54     beq .EnableMissile1
55 .DisableMissile1
56     lda #0
57     jmp .Finish1
58 .EnableMissile1
59     lda #2

```

```

60 .Finish1
61     sta ENAM1
62     cpx #DirtYStart
63     bmi .innerLoop
64
65
66     ; Next Scanline - draw playfield data - Originally by Steven
        Hugg, alterations by Matthew Prickett
67     ; -----
68     sta WSYNC
69     dec DirtHeightCounter
70
71     lda Dirt+NumDirtRows*0,y ; 0+12*0,y
72     sta PF0
73     lda Dirt+NumDirtRows*1,y ; 0+12*1,y
74     sta PF1
75     lda Dirt+NumDirtRows*2,y
76     sta PF2      ; 21 machine cycles - 1 off HBLANK
77
78     inx      ; Increment scanline counter
79
80     lda Dirt+NumDirtRows*3,y
81     sta PF0
82     lda Dirt+NumDirtRows*4,y
83     sta PF1
84     lda Dirt+NumDirtRows*5,y
85     sta PF2      ; 21 machine cycles
86
87     lda DirtHeightCounter
88     bne .innerLoop
89     beq DrawLoop      ; Total 55 machine cycles used here, 21
        machine cycles free
90
91 DoneDirtDraw
92     sta WSYNC
93
94     ; Turn off missiles and playfield data
95     lda #0
96     sta PF0
97     sta PF1
98     sta PF2
99     sta ENAM0
100    sta ENAM1

```

Listing 5: Main kernel for the game, handling draw the dirt in an asymmetrical way and drawing the missile beam or reticle

```

1 ; x - contains the player missile and player score to add to
2 MissileCollision subroutine
3     lda CXM0FB,x
4     sta Temp
5     lda #%11000000
6     bit Temp
7     beq .nocollision
8 .collision
9     txa
10    tay    ; Move the player we are looking at into the Y register
11    pha    ; Save the contents of the X register
12
13    lda MissileY0,y ; A contains missile Y coordinate
14    ldx XPos0,y ; X contains missile X coordinate
15
16    jsr CleanDirt
17    bmi .nocollision2
18
19    pla
20    tax    ; Load the player we are looking at into the X register
21
22    lda RowCleaned
23    bne .NoAddScore
24
25    txa
26    pha
27
28    ldy #3
29    ldx #0
30    jsr PlaySound
31
32    pla
33    tax
34
35    ; Add to the player score if a PF was cleaned
36    clc
37    sed
38    lda Score0,x
39    adc #1
40    sta Score0,x
41    cld
42 .NoAddScore
43     jmp .nocollision
44 .nocollision2

```

```

45     pla
46     tax
47 .nocollision
48     rts

```

Listing 6: Code handling the collisions between missiles and dirt, determining whether dirt should be destroyed, audio should play and points awarded

```

1 ; Originally by Steven Hugg, altered by Matthew Prickett
2
3 ; Subroutine to try to clear a bit of dirt at a given X-Y
  coordinate.
4 ; X contains the X coordinate.
5 ; A contains the Y coordinate.
6 ; On return, A = -1 if no dirt was present,
7 ; otherwise A = Y offset (0-brickheight-1) of dirt hit.
8 CleanDirt
9     ldy #$ff
10    sec
11    sbc DirtYStart ; subtract top Y of dirt field
12 ; Divide by dirt height
13 DivideRowLoop
14     iny
15     sbc #DirtHeight*2 ; #
16     bcs DivideRowLoop ; loop until < 0
17     cpy #NumDirtRows ; #
18     bcs NoBrickFound
19 ; Now that we have the line, get byte and bit offset for brick
20     clc
21     adc #DirtHeight*2 ; #
22     pha ; save the remainder to return as result
23     txa
24     clc
25     adc #3 ; adjust because SetHorizPos is off by a few pixels
26     lsr
27     lsr ; divide X coordinate by 4
28     tax ; transfer brick column to X
29     tya ; load brick row # in A
30     clc
31     adc PFOfsTable,x ; add offset
32     tay
33     lda PFMaskTable,x
34     eor #$ff
35     and Dirt,y
36     cmp Dirt,y ; was there a change?
37     beq NoBrickFound2 ; no, so return -1 as result

```

```

38         sta Dirt,y
39         sta RowCleaned
40         pla    ; return remainder as result
41         rts
42 NoBrickFound2
43     pla    ; pull the remainder, but ignore it
44 NoBrickFound
45     lda #$FF ; return -1 as result
46     rts

```

Listing 7: Subroutine that calculates what bit in memory to zero out, originally by Steven Hugg [18]

```

1 MoveJoystick0 subroutine
2 ; Move horizontally
3     ldx XPos0
4     lda #%01000000 ;Left?
5     bit SWCHA
6     bne .SkipMoveLeft
7     cpx #1
8     bcc .SkipMoveLeft
9     dex
10 .SkipMoveLeft
11     lda #%10000000 ;Right?
12     bit SWCHA
13     bne .SkipMoveRight
14     cpx #153
15     bcs .SkipMoveRight
16     inx
17 .SkipMoveRight
18     stx XPos0
19 ; Move Vertically
20     ldy MissileY0
21     lda #%00010000 ;Up?
22     bit SWCHA
23     bne .SkipMoveUp
24     cpy #1
25     bcc .SkipMoveUp
26     dey
27     dey
28 .SkipMoveUp
29     lda #%00100000 ;down?
30     bit SWCHA
31     bne .SkipMoveDown
32     cpy #DirtYStart+DirtHeight*2*NumDirtRows
33     bcs .SkipMoveDown

```

```

34     iny
35     iny
36 .SkipMoveDown
37     sty MissileY0
38     rts

```

Listing 8: Complete Joystick input handling for movement and aiming

```

1  ; Returns a negative number if not all clean
2 TestAllClean subroutine
3     ldx #$FF
4 .CheckRow
5     inx
6     lda Dirt+NumDirtRows*0,x
7     and #$F0
8     bne .NotClean
9
10    lda Dirt+NumDirtRows*1,x
11    bne .NotClean
12
13    lda Dirt+NumDirtRows*2,x
14    bne .NotClean
15
16    lda Dirt+NumDirtRows*3,x
17    and #$F0
18    bne .NotClean
19
20    lda Dirt+NumDirtRows*4,x
21    bne .NotClean
22
23    lda Dirt+NumDirtRows*5,x
24    bne .NotClean
25
26    cpx #NumDirtRows-1
27    bne .CheckRow
28    rts
29 .NotClean
30    lda #$FF
31    rts

```

Listing 9: The algorithm to loop through dirt memory to test if it is clear

```

1  ; Originally by Steven Hugg from the book "Making Games for the
   ; Atari 2600"
2
3  ; The X register contains the index of the desired object:
4  ; X=0: player 0

```

```

5 ; X=1: player 1
6 ; X=2: missile 0
7 ; X=3: missile 1
8 ; X=4: ball
9 SetHorizPos
10     sta WSYNC ; start a new line
11         bit 0    ; waste 3 cycles
12     sec      ; set carry flag
13 DivideLoop
14     sbc #15   ; subtract 15
15     bcs DivideLoop ; branch until negative
16     eor #7    ; calculate fine offset
17     asl
18     asl
19     asl
20     asl
21     sta RESP0,x ; fix coarse position
22     sta HMP0,x  ; set fine offset
23     rts      ; return to caller

```

Listing 10: Algorithm to correctly position any object at a given X-coordinate originally developed by Steven Hugg [18]

```

1 ; X contains ProcData index (0,1,2,3)
2 ; Y contains row index (0-11)
3 ; A will contain output
4 GetProcData subroutine
5     bne .next1
6     lda ProcData+NumDirtRows*0,y
7     jmp .load
8 .next1
9     dex
10    bne .next2
11    lda ProcData+NumDirtRows*1,y
12    jmp .load
13 .next2
14    dex
15    bne .next3
16    lda ProcData+NumDirtRows*2,y
17    jmp .load
18 .next3
19    lda ProcData+NumDirtRows*3,y
20 .load
21    rts
22
23 ; X contains ProcData index (0,1,2,3)

```

```

24 ; Y contains row index (0-11)
25 ; A will contain output
26 GetHalfProcData subroutine
27     bne .next1
28     lda HalfProcData+NumDirtRows*0,y
29     jmp .load
30 .next1
31     dex
32     bne .next2
33     lda HalfProcData+NumDirtRows*1,y
34     jmp .load
35 .next2
36     dex
37     bne .next3
38     lda HalfProcData+NumDirtRows*2,y
39     jmp .load
40 .next3
41     lda HalfProcData+NumDirtRows*3,y
42 .load
43     rts

```

Listing 11: Subroutines that returns the data for a particular row based on the given value

```

1 LoadProcedural subroutine
2     inc RandomNum
3     ldx RandomNum
4     lda DrawLoop,x
5     sta Temp
6
7     ldy #$FF
8 .ProceduralLoop
9     iny
10
11     lda Temp
12     and #$03
13     pha
14     tax
15     jsr GetProcData
16     sta Dirt+NumDirtRows,y
17     pla
18     tax
19     jsr GetHalfProcData
20     sta Dirt+NumDirtRows*0,y
21
22     lda Temp
23     and #$0C

```

```

24     lsr
25     lsr
26     tax
27     jsr GetProcData
28     sta Dirt+NumDirtRows*2,y
29
30     cpy #NumDirtRows-1
31     bne .ProceduralLoop
32     rts
33
34 LoadProcedural1 subroutine
35     ldx RandomNum
36     lda DrawLoop,x
37     sta Temp
38
39     ldy #$FF
40 .ProceduralLoop
41     iny
42
43     lda Temp
44     and #$30
45     lsr
46     lsr
47     lsr
48     lsr
49     tax
50     jsr GetProcData
51     sta Dirt+NumDirtRows*4,y
52
53     lda Temp
54     and #$C0
55     rol
56     rol
57     rol
58     pha
59     tax
60     jsr GetProcData
61     sta Dirt+NumDirtRows*5,y
62     pla
63     tax
64     jsr GetHalfProcData
65     sta Dirt+NumDirtRows*3,y
66
67     cpy #NumDirtRows-1
68     bne .ProceduralLoop

```

```
69      rts
```

Listing 12: Subroutine parts to completely load the dirt memory with data. Split into two to avoid flicker issue

```
1 LoadFull subroutine
2     ldx #$FF
3 .FullLoop
4     inx
5     lda #$F0
6     sta Dirt+NumDirtRows*0,x
7     sta Dirt+NumDirtRows*3,x
8
9     lda #$FF
10    sta Dirt+NumDirtRows*1,x
11    sta Dirt+NumDirtRows*2,x
12    sta Dirt+NumDirtRows*4,x
13    sta Dirt+NumDirtRows*5,x
14
15    cpx #NumDirtRows-1
16    bne .FullLoop
17
18    rts
```

Listing 13: Subroutine that populates all of the dirt memory with data

```
1 LoadDirt subroutine
2     lda #%00001000
3     bit SWCHB
4
5     bne .procedural
6     lda SWCHB
7     bmi .random
8     jsr LoadFull
9     jmp .return
10 .random
11     jsr LoadRandom
12     jmp .return
13 .procedural
14     cpx #0
15     bne .secondProc
16     jsr LoadProcedural
17     jmp .return
18 .secondProc
19     jsr LoadProcedural1
20 .return
```

21 rts

Listing 14: Code that handles reading the input switches to load the dirt in different ways

```
1 ; Make the missile big if it is Reticle Mode for player 0
2   lda MissileVel0
3   bmi CursorSize0
4   lda #0
5   jmp StoreSize0
6 CursorSize0
7   lda #%000011000
8 StoreSize0
9   sta NUSIZ0
10
11 ; Recolour missile for either Reticle Mode or Stream Mode
12   lda MissileVel0
13   bmi ReticleColour0
14   lda #$af
15   jmp StoreColour0
16 ReticleColour0
17   lda #Player0Colour
18 StoreColour0
19   sta MissileColour0
```

Listing 15: Code that handles setting the missile size and colour depending on if it is in reticle mode or beam mode

```
1 AnimatePlayer0 subroutine
2   lda #$C0
3   and SWCHA
4   cmp #$C0
5   beq .idle
6
7   lda FrameCounter
8   lsr
9   lsr
10  lsr
11  lsr
12
13  bcs .SetFrame2
14
15  lda #<Frame1
16  sta SpritePtr0
17  lda #>Frame1
18  sta SpritePtr0+1
19  jmp .return
20 .SetFrame2
```

```

21     lda #<Frame2
22     sta SpritePtr0
23     lda #>Frame2
24     sta SpritePtr0+1
25     jmp .return
26 .idle
27     lda #<Frame0
28     sta SpritePtr0
29     lda #>Frame0
30     sta SpritePtr0+1
31 .return
32     rts

```

Listing 16: Code that handles player animation for the walking cycle

```

1 P0CalculatePercentage subroutine
2 ; (NumDirtP0,2x) * 100 / NumDirtBlocks
3 ; First need to multiply x by 100 = 100x = 4( 4(4x + 2x) + x) =
4
5     dec P0CalcFrame
6     lda P0CalcFrame
7     cmp #2
8     bne .Next1
9     jsr P0MultiplyBy100
10    jmp .return
11 .Next1 cmp #1
12    bne .Next0
13    jsr P0DivideByNumDirt
14    jmp .return
15 .Next0
16    jsr P0UpdateScore
17    lda #3
18    sta P0CalcFrame
19 .return
20    rts
21
22    ;2 - Multiply
23    ;1 - Divide
24    ;0 - Update
25
26
27 ; x contains the player we are looking at (either 0 or 2)
28 P0MultiplyBy100 subroutine
29
30    ; Begin Multiplication by 100
31    lda NumDirtP0

```

```

32     sta CalcBuffer0
33     lda NumDirtP0+1
34     sta CalcBuffer0+1    ; load NumDirtP0 into CalcBuffer0 ;14
                          mcs ; 24mcs
35
36     asl CalcBuffer0
37     rol CalcBuffer0+1    ; CalcBuffer0 = 2*NumDirtP0,2x
38     asl CalcBuffer0
39     rol CalcBuffer0+1 ; CalcBuffer0 = 4*NumDirtP0,2x    ;20mcs ;
                          44mcs
40
41     clc
42     lda NumDirtP0
43     adc CalcBuffer0
44     sta CalcBuffer0
45     lda NumDirtP0+1
46     adc CalcBuffer0+1
47     sta CalcBuffer0+1 ; CalcBuffer0 = 5*NumDirtP0,2x    ;22mcs ;
                          66mcs
48
49     asl CalcBuffer0
50     rol CalcBuffer0+1 ; CalcBuffer0 = 10*NumDirtP0,2x    ;10mcs
                          ; 76mcs
51
52     lda CalcBuffer0
53     sta Temp
54     lda CalcBuffer0+1
55     sta Temp+1    ; Temp = 10*NumDirtP0,2x    ;12mcs ; 88mcs
56
57
58     asl CalcBuffer0
59     rol CalcBuffer0+1 ; CalcBuffer0 = 20*NumDirtP0,2x
60     asl CalcBuffer0
61     rol CalcBuffer0+1 ; CalcBuffer0 = 40*NumDirtP0,2x    ;20mcs
                          ; 108mcs
62
63     clc
64     lda Temp
65     adc CalcBuffer0
66     sta CalcBuffer0
67     lda Temp+1
68     adc CalcBuffer0+1
69     sta CalcBuffer0+1 ; CalcBuffer0 = 50*NumDirtP0,2x    ;22mcs
                          ; 130mcs
70

```

```

71         asl CalcBuffer0
72         rol CalcBuffer0+1 ; CalcBuffer0 = 100*NumDirtP0,2x ;10mcs
           ; 140mcs
73
74         rts
75
76 P0DivideByNumDirt subroutine
77     lda CalcBuffer0
78     beq .return
79     lda #0
80     sta Temp      ; Temp will store the remainder
81     sta Temp+1    ;8mcs ; 148mcs
82
83     ldx #16      ; 16 bits ;2mcs ; 150mcs
84 .L1 asl CalcBuffer0
85     rol CalcBuffer0+1
86     rol Temp
87     rol Temp+1
88     lda Temp
89     sec
90     sbc NumDirtBlock
91     tay
92     lda Temp+1
93     sbc NumDirtBlock+1
94     bcc .L2
95     sta Temp+1
96     sty Temp
97     inc CalcBuffer0
98 .L2     dex
99     bne .L1
100 .return
101     rts
102
103 P0UpdateScore subroutine
104     clc
105
106     lda #0
107     sta Score0
108
109     lda CalcBuffer0
110     sed
111     adc Score0
112     sta Score0
113     cld
114

```

115

```
rts
```

Listing 17: Attempted inclusion of percentage calculation that was not working correctly and was eventually scrapped

Appendix C: Test Suites

Movement

Test Case	SWCHA	XPos0	Expected Output	Actual Output
1	P0 left	0	Player 0 does not move	0 (Figure 17)
2	P0 left	40	Player 0 moves left	39 (Figure 18)
3	P0 right	153	Player 0 does not move	153 (Figure 21)
4	P0 right	40	Player 0 moves right	41 (Figure 20)
5	No-Input	40	Player 0 does not move	40 (Figure 19)
6	P1 left	40	Player 0 does not move	40 (Figure 23)
7	P1 right	40	Player 0 does not move	40 (Figure 24)

Table 7: Test Suite for Player 0 movement

Test Case	SWCHA	XPos1	Expected Output	Actual Output
1	P1 left	0	Player 1 does not move	0 (Figure 25)
2	P1 left	112	Player 1 moves left	111 (Figure 23)
3	P1 right	153	Player 1 does not move	153 (Figure 25)
4	P1 right	112	Player 1 moves right	113 (Figure 24)
5	No-Input	112	Player 1 does not move	112 (Figure ??)
6	P0 left	112	Player 1 does not move	112 (Figure 18)
7	P0 right	112	Player 1 does not move	112 (Figure 20)

Table 8: Test Suite for Player 1 movement

Aiming

Test Case	SWCHA	MissileY0	Expected Output	Actual Output
1	P0 up	0	Missile 0 does not move	0 (Figure 26)
2	P0 up	70	Missile 0 moves Up	68 (Figure 27)
3	P0 down	128	Missile 0 does not move	128 (Figure 30)
4	P0 down	70	Missile 0 moves up	72 (Figure 29)
5	No-input	70	Missile 0 does not move	70 (Figure 28)
6	P1 up	70	Missile 0 does not move	70 (Figure 32)
7	P1 down	70	Missile 0 does not move	70 (Figure 33)

Table 9: Test Suite for Player 0 aiming

Test Case	SWCHA	MissileY1	Expected Output	Actual Output
1	P1 up	0	Missile 1 does not move	0 (Figure 31)
2	P1 up	70	Missile 1 moves Up	68 (Figure 32)
3	P1 down	128	Missile 1 does not move	128 (Figure 34)
4	P1 down	70	Missile 1 moves up	72 (Figure 33)
5	No-input	70	Missile 1 does not move	70 (Figure 28)
6	P0 up	70	Missile 1 does not move	70 (Figure 27)
7	P0 down	70	Missile 1 does not move	70 (Figure 29)

Table 10: Test Suite for Player 1 aiming

Shooting

Test Case	INPT4 (P0)	Expected Output	Actual Output
1	Button Press	A beam can be seen from Player 0	A beam emits from Player 0 (Figure 36)
2	Button Depress	A reticle and no beam for Player 0	A blue coloured reticle, no beam (Figure 35)

Table 11: Test Suite for Player 0 Shooting

Test Case	INPT5 (P1)	Expected Output	Actual Output
1	Button Press	A beam can be seen from Player 1	A beam emits from Player 1 (Figure 37)
2	Button Depress	A reticle and no beam for Player 1	A red coloured reticle, no beam (Figure 35)

Table 12: Test Suite for Player 1 Shooting

Dirt Collisions

Test Case	INPT4 (P0)	XPos0	MissileY0	Dirt Present	Expected Output	Actual Output
1	Button Press	40	70	True	Dirt only at location is destroyed	Dirt only at location is destroyed (Figure 39)
2	Button Depress	40	70	True	Dirt state unchanged	Dirt is unchanged (Figure 41)
3	Button Press	40	70	False	Dirt state unchanged	Dirt is unchanged (Figure 40)
4	Button Depress	40	70	False	Dirt state unchanged	Dirt is unchanged (Figure 38)

Table 13: Test Suite for Player 0 Cleaning

Test Case	INPT5 (P1)	XPos1	MissileY1	Dirt Present	Expected Output	Actual Output
1	Button Press	112	70	True	Dirt only at location is destroyed	Dirt only at location is destroyed (Figure 42)
2	Button Depress	112	70	True	Dirt state unchanged	Dirt is unchanged (Figure 44)
3	Button Press	112	70	False	Dirt state unchanged	Dirt is unchanged (Figure 43)
4	Button Depress	112	70	False	Dirt state unchanged	Dirt is unchanged (Figure 38)

Table 14: Test Suite for Player 1 Cleaning

Test Case	Playfield Segment	Row	Expected Output	Actual Output
1	PF0 (0)	0	Dirt Cleaned	Dirt Cleaned
2	PF1 (0)	1	Dirt Cleaned	Dirt Cleaned
3	PF2 (0)	2	Dirt Cleaned	Dirt Cleaned
4	PF0 (1)	3	Dirt Cleaned	Dirt Cleaned
5	PF1 (1)	4	Dirt Cleaned	Dirt Cleaned
6	PF2 (1)	5	Dirt Cleaned	Dirt Cleaned
7	PF0 (0)	6	Dirt Cleaned	Dirt Cleaned
8	PF1 (0)	7	Dirt Cleaned	Dirt Cleaned
9	PF2 (0)	8	Dirt Cleaned	Dirt Cleaned
10	PF0 (1)	9	Dirt Cleaned	Dirt Cleaned
11	PF1 (1)	10	Dirt Cleaned	Dirt Cleaned
12	PF2 (1)	11	Dirt Cleaned	Dirt Cleaned

Table 15: Test Suite for cleaning whole segments of rows and columns (see Figure 45 for complete test).

Scoring

Test Case	INPT4	INPT5	PF Segment	Row	Expected Output	Actual Output
1	1	0	PF0 (0)	0	Score0 increase	Score0 from 0 to 1
2	0	1	PF1 (0)	11	Score1 increase	Score1 from 0 to 1
3	1	1	PF2 (0)	5	Score0 increase	Score0 from 1 to 2
4	0	1	PF0 (1)	11	Score1 increase	Score1 from 1 to 2
5	1	1	PF1 (1)	5	Score0 increase	Score0 from 2 to 3
6	1	0	PF2 (1)	0	Score0 increase	Score0 from 3 to 4

Table 16: Test Suite for adding score to the player who cleaned the last bit in a segment (see Figure 46)

Test Case	Dirt Left	Expected Output	Actual Output
1	440	Game continues	Game continued (Figure 19)
2	40	Game continues	Game continued (Figure 48)
3	1	Game continues	Game continued (Figure 47)
4	0	Game ends	Game ended (Figure 49)

Table 17: Test Suite for checking that all segments are cleaned (TestAllClean subroutine)

Test Case	Score0	Score1	Expected Output	Actual Output
1	72	0	Player 0 win	Player 0 win animation (Figure 49)
2	0	72	Player 1 win	Player 1 win animation (Figure 50)
3	36	36	Draw	Draw animation

Table 18: Test Suite for winning

Test Case	Score0 Increase	Score1 Increase	Expected Output	Actual Output
1	False	False	No audio	No audio
2	True	False	Audio	Audio
3	False	True	Audio	Audio
4	True	True	Audio	Audio

Table 19: Test Suite for testing that the ding sound audio plays when only when either player clears a segment.

Loading Dirt

Test Case	BW	P1 Difficulty	Expected Output	Actual Output
1	0	0	Full dirt canvas	All dirt loaded (Figure 28)
2	0	1	Random dirt canvas	Random pattern of dirt (Figure 51)
3	1	0	Procedural Generation	Procedural dirt loading (Figure 35)
4	1	1	Procedural Generation	Procedural dirt loading

Table 20: Test Suite for switching between different load modes

Test Case	AreaNum	Expected BG	Expected FG	Actual BG	Actual FG
1	0	\$ba	\$54	\$ba	\$54 (Figure 35)
2	1	\$1a	\$a0	\$1a	\$a0 (Figure 52)
3	2	\$61	\$1e	\$61	\$1e (Figure 53)
4	3	\$ff	\$d0	\$ff	\$d0 (Figure 54)

Table 21: Test Suite for changing colours every new round

Test Case	Data Enum	Row	Expected Output	Actual Output
1	0 (House)	0	00011000	00011000
2	1 (TwoStorey)	11	11101110	11101110
3	2 (Car)	5	00000000	00000000
4	3 (Office)	6	01111111	01111111

Table 22: Test Suite for looking up 8-bit segment procedural loading

Test Case	Data Enum	Row	Expected Output	Actual Output
1	0 (Small House)	11	11110000	11110000
2	1 (Bin)	6	00000000	00000000
3	2 (House2)	5	01110000	01110000
4	3 (Flag)	0	00000000	00000000

Table 23: Test Suite for looking up 4-bit segment procedural loading

Test Case	Procedural-Number	Expected Output	Actual Output
1	00111111	[3][3][3][0][3][0]	[3][3][3][0][3][0] (Figure 55)
2	01101010	[2][2][2][1][2][1]	[2][2][2][1][2][1] (Figure 57)
3	10010101	[1][1][1][2][1][2]	[1][1][1][2][1][2] (Figure 57)
4	11000000	[0][0][0][3][0][3]	[0][0][0][3][0][3] (Figure 58)

Table 24: Test Suite for procedural generation loading

Animation and Audio

Test Case	SWCHA	Expected Output	Actual Output
1	P0 left	Player 0 movement animation plays	Player 0 movement animation plays
2	P0 right	Player 0 movement animation plays	Player 0 movement animation plays
3	No-Input	Player 0 movement animation does not play	Player 0 idle sprite, no movement animation
4	P1 left	Player 0 movement animation does not play	Player 0 idle sprite, no movement animation
5	P1 right	Player 0 movement animation does not play	Player 0 idle sprite, no movement animation

Table 25: Test Suite for Player 1 movement animation

Test Case	SWCHA	Expected Output	Actual Output
1	P1 left	Player 1 movement animation plays	Player 1 movement animation plays
2	P1 right	Player 1 movement animation plays	Player 1 movement animation plays
3	No-Input	Player 1 movement animation does not play	Player 1 idle sprite, no movement animation
4	P0 left	Player 1 movement animation does not play	Player 1 idle sprite, no movement animation
5	P0 right	Player 1 movement animation does not play	Player 1 idle sprite, no movement animation

Table 26: Test Suite for Player 1 movement animation

Test Case	INPT4	INPT5	Expected Output	Actual Output
1	0	0	No audio	No audio
2	1	0	Audio	Audio
3	0	1	Audio	Audio
4	1	1	Audio	Audio

Table 27: Test Suite for shooting audio

Test Case	Game State	Expected Output	Actual Output
1	Playing	No audio	No audio
2	Win	Audio	Audio

Table 28: Test Suite for game win music jingle

Appendix D: Test Images

Movement



Figure 17: A screenshot showing P0 going no further than the left edge.

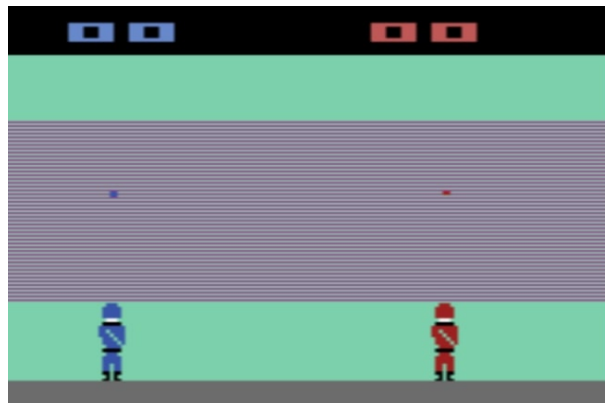


Figure 18: A screenshot showing P0 going left.

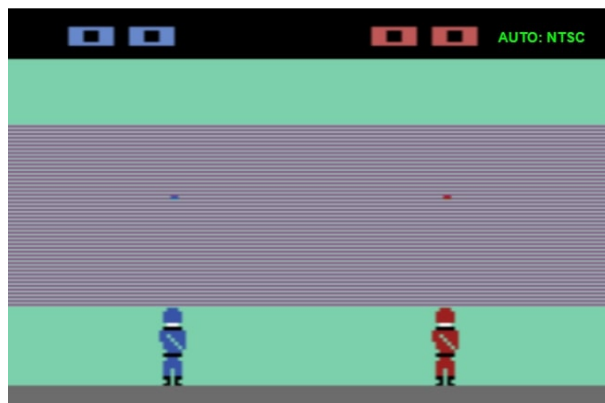


Figure 19: A screenshot showing no input for either player.

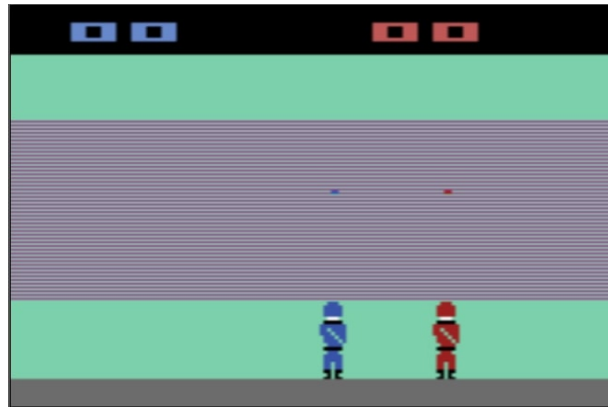


Figure 20: A screenshot showing P0 going right.



Figure 21: A screenshot showing P0 going no further than the right edge.

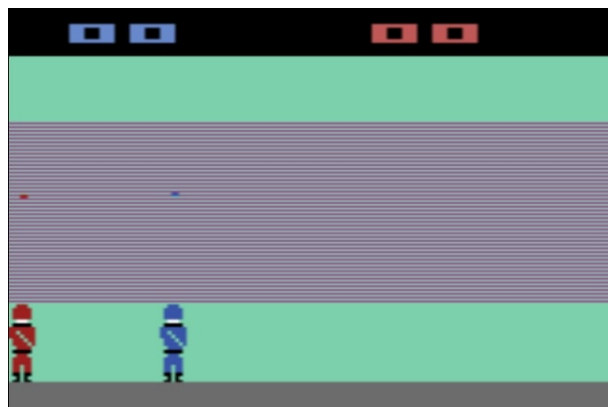


Figure 22: A screenshot showing P1 going no further than the left edge.

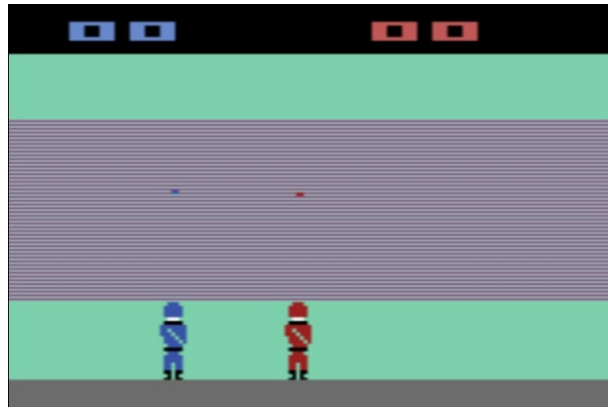


Figure 23: A screenshot showing P1 going left.

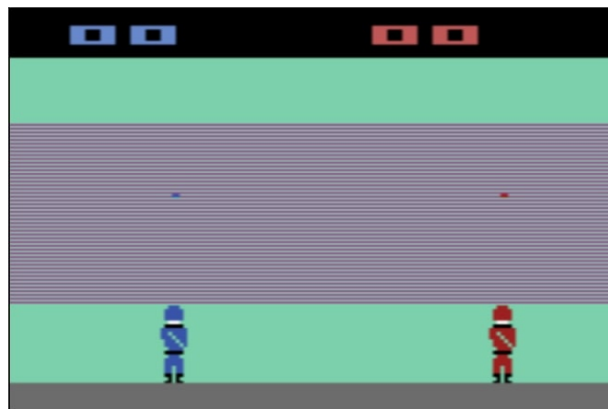


Figure 24: A screenshot showing P1 going right.

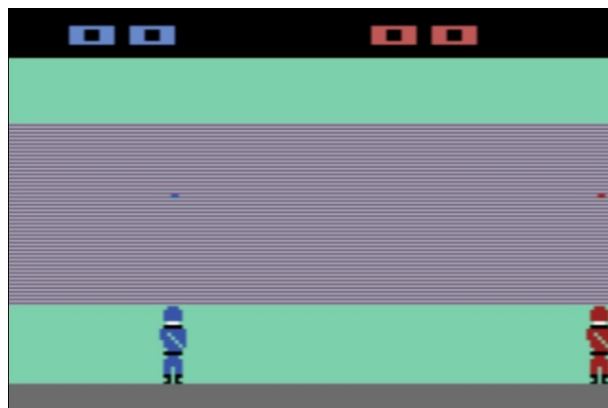


Figure 25: A screenshot showing P1 going no further than the right edge.

Aiming

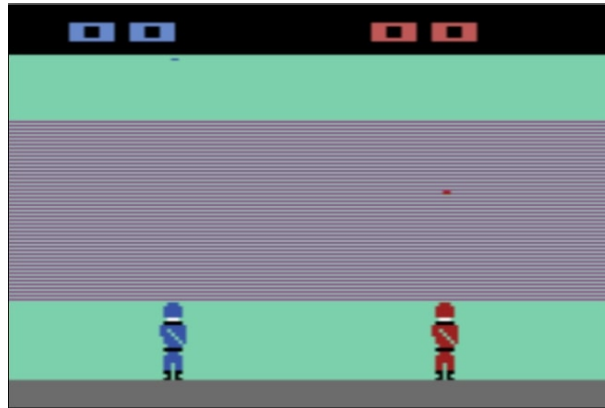


Figure 26: A screenshot showing P0 reticle going no further top of play area.

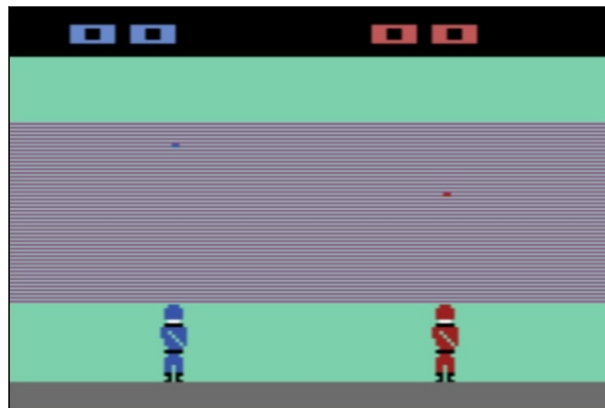


Figure 27: A screenshot showing P0 reticle going up.

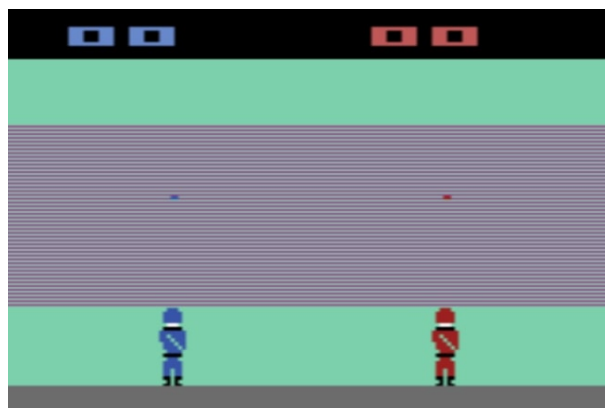


Figure 28: A screenshot showing P0 reticles not moving with no input.

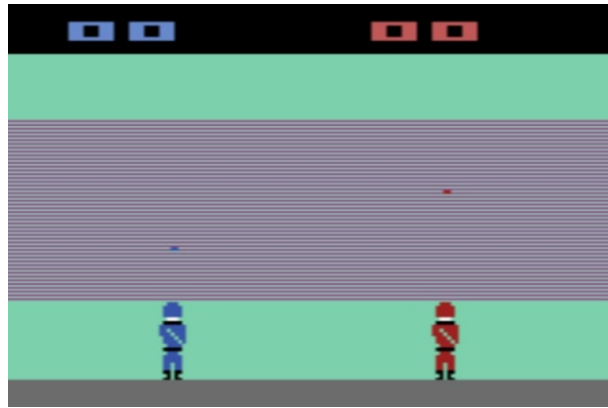


Figure 29: A screenshot showing P0 reticle going down.

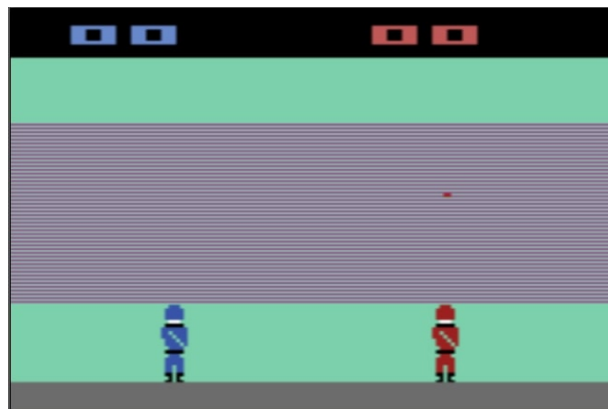


Figure 30: A screenshot showing P0 reticle going no further then the bottom of the play area.

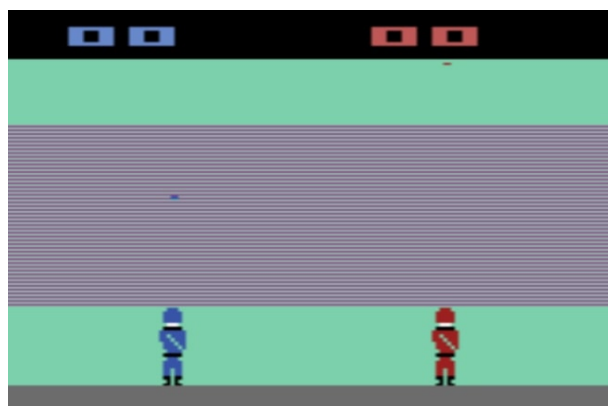


Figure 31: A screenshot showing P1 reticle going no further top of play area.

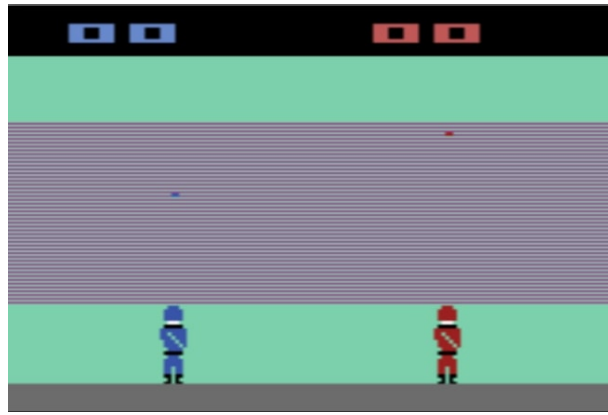


Figure 32: A screenshot showing P1 reticle going up.



Figure 33: A screenshot showing P1 reticle going down.



Figure 34: A screenshot showing P1 reticle going no further than the bottom of the play area.

Shooting

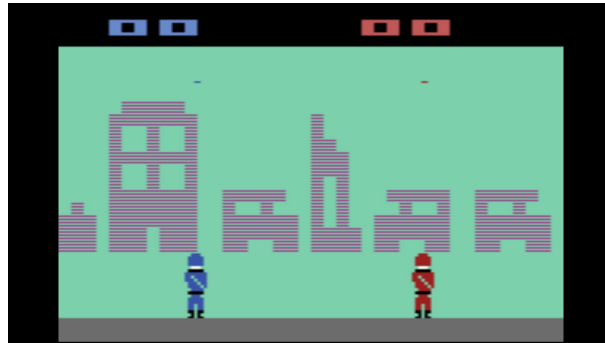


Figure 35: A screenshot showing neither player firing their washer.



Figure 36: A screenshot showing Player 0 firing their washer.



Figure 37: A screenshot showing Player 1 firing their washer.

Dirt Collisions

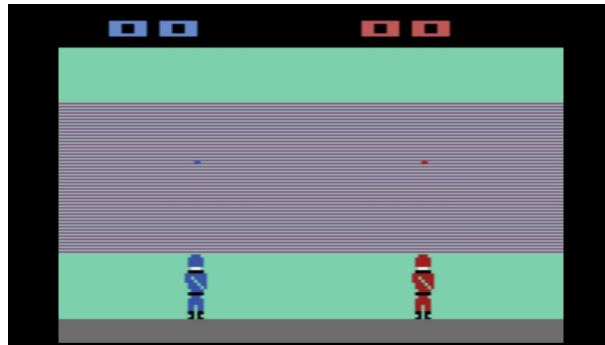


Figure 38: A screenshot showing neither players in firing mode and causing a collision.

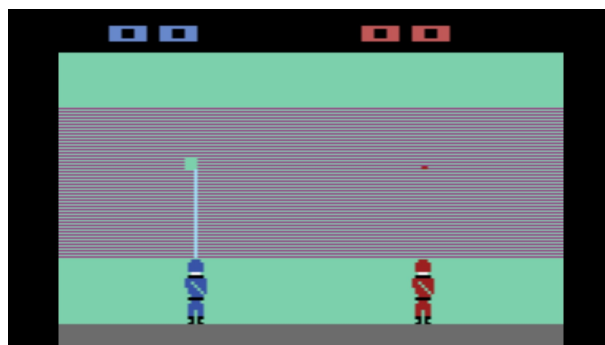


Figure 39: A screenshot showing Player 0 firing and causing a dirt piece to be washed.

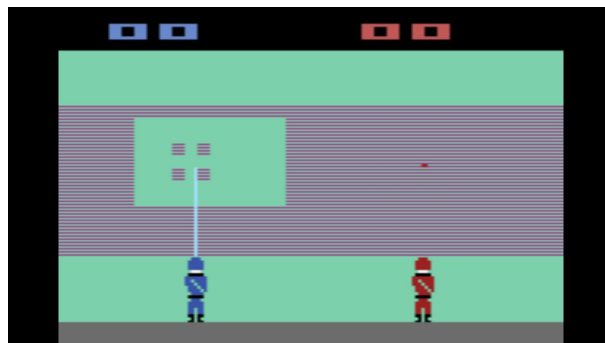


Figure 40: A screenshot showing Player 0 firing and not causing a dirt piece to be washed.

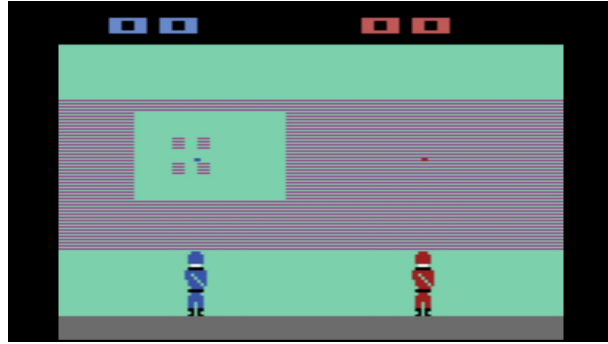


Figure 41: A screenshot showing Player 0 not firing and not causing a collision.

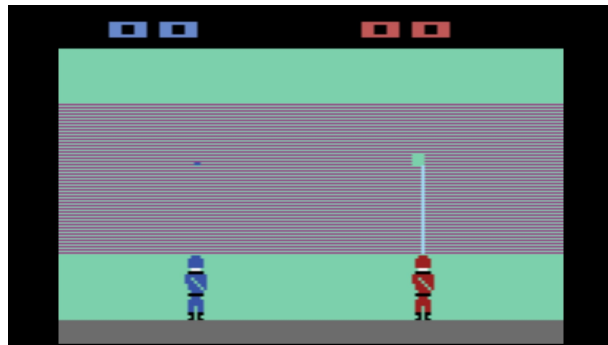


Figure 42: A screenshot showing Player 1 firing and causing a dirt piece to be washed.

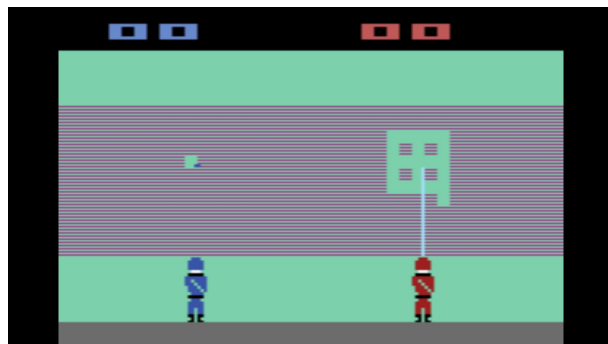


Figure 43: A screenshot showing Player 1 firing and not causing a dirt piece to be washed.

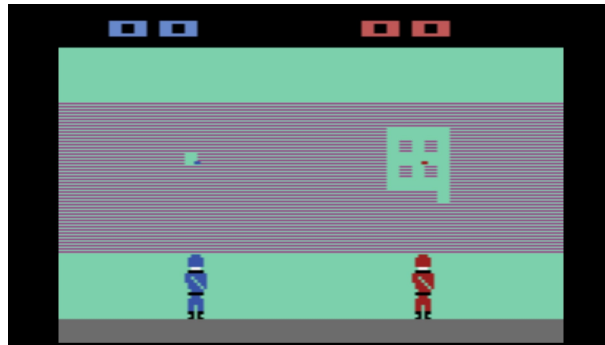


Figure 44: A screenshot showing Player 1 not firing and not causing a collision.

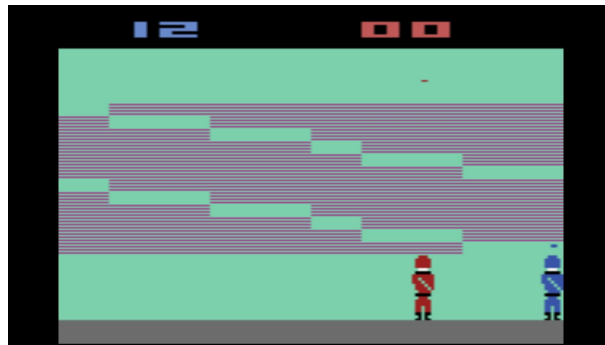


Figure 45: A screenshot showing every row and column can be washed.

Scoring



Figure 46: A screenshot of the complete test showing score is only added when a whole playfield segment is cleaned.



Figure 47: A screenshot showing one dirt piece left and the game continuing



Figure 48: A screenshot showing 40 dirt pieces left and the game continuing



Figure 49: A screenshot showing the P0 win animation



Figure 50: A screenshot showing the P1 win animation

Loading Dirt

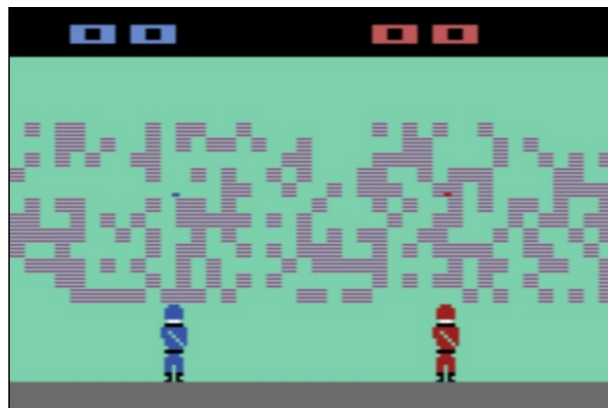


Figure 51: A screenshot showing random loading when in BW mode and P1 expert mode



Figure 52: A screenshot showing the colour screen for area two

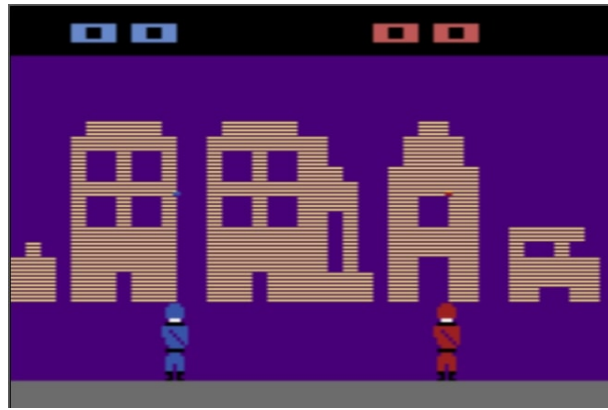


Figure 53: A screenshot showing the colour screen for area three



Figure 54: A screenshot showing the colour screen for area four



Figure 55: A screenshot showing procedural generation test case 1

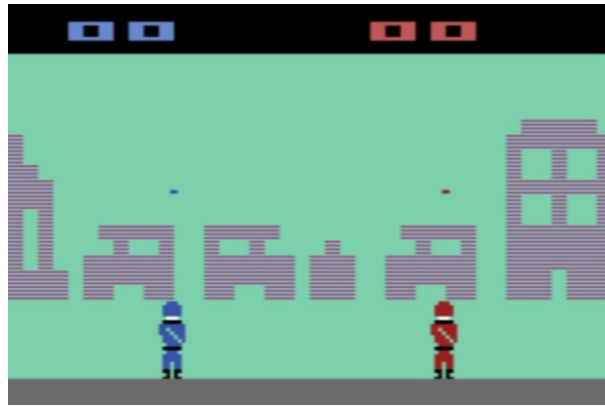


Figure 56: A screenshot showing procedural generation test case 2



Figure 57: A screenshot showing procedural generation test case 3



Figure 58: A screenshot showing procedural generation test case 4

Appendix E: Miscellaneous

Original Specification

1. The game must fall within the hardware constraints of the Atari 2600 system
 - ROM space must not exceed 8K, i.e. 2x4K bank-switching.
 - RAM usage must not exceed 128 bytes.
 - Graphics comply to the available resources for the TIA to display.
2. The graphics should feature recognisable elements from Power Wash Simulator.
 - The player model and power washer should feature to some capacity.
 - The logo of the game must be displayed on screen before the player starts.
3. Playing the game must be smooth and not stutter.
 - Sprite flickering must be kept to a minimum like 'Pac-Man 4K'.
4. The game must have a single player mode.
 - The objective is to gain as much score as possible within a short period (e.g. two minutes).
 - Score is obtained by cleaning objects within a room.
 - Cleaning a segment of an object will contribute to score.
 - Completely cleaning an object will give additional score.
 - The player should be able to move to different rooms where new objects are able to be cleaned.
 - Objects in the previous room do not need to be cleaned entirely before the player can move to the new room.
5. The game should have a two-player mode.
 - The objective is to have more score than your opponent within the same period as single-player.
 - Basic game mechanics work similarly to single player, except for being able to move to a new room:
 - Both players are only able to go to a new room once the current room and objects in it has been entirely cleaned.
 - Each player may interfere with the other by firing at their opponent to lock out their movement controls and displace them on screen.
6. The game should have intuitive controls.
 - Moving the joystick in a direction will move the character in the corresponding direction on screen.

- Hitting the button on the joystick will cause the power washer to fire.
7. the game should contain multiple attachments that randomly appear throughout a game:
- Attachments refer to the different types of "nozzles" that the power washer can have.
 - Attachments will change how the power washer fires.
 - One attachment will cause the power washer to fire quicker but targets a smaller area.
 - Another attachment should fire slower but hits a larger area.
8. Cleaning objects should cause them to change colour
- This is so the player can identify cleaned and not cleaned objects easily.
 - Cleaning a segment of an object will contribute to the score.
 - Completely cleaning an object will give bonus score.
 - In two player, whichever player completes the object obtains the score.
 - Completing a cleaned object will cause the object to "flash" indicating it is complete.
 - Accompanying this should be a sound effect.

Diagram of Playfield Graphics

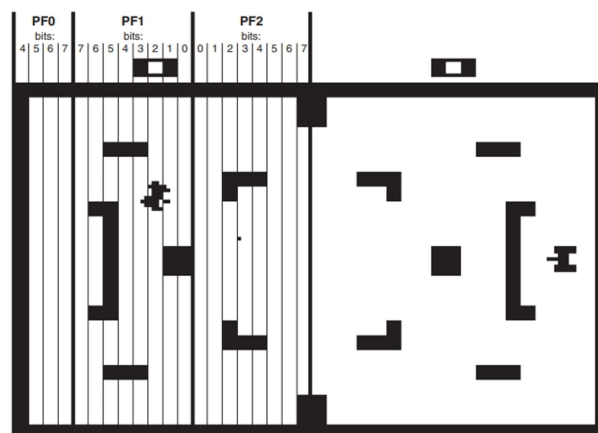


Figure 59: A diagram of the Playfield Graphics, showing that only 20 bits store the data. These can be repeated or mirrored [23]